Introduction on UML for Industrial Systems

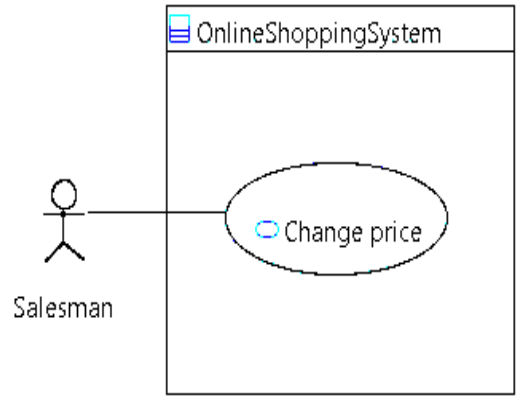
Ansgar Radermacher / Asma Smaoui
ansgar.radermacher@cea.fr

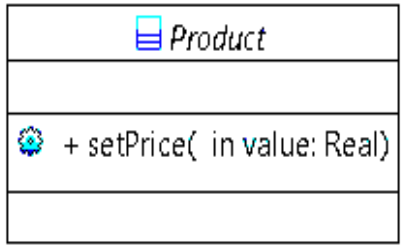
Acknowledgments – contains material from my CEA colleagues
Shuai Li, Jérémie Tatibouët, François Terrier, Sébastien Gérard

Behavior diagrams – interactions & activities

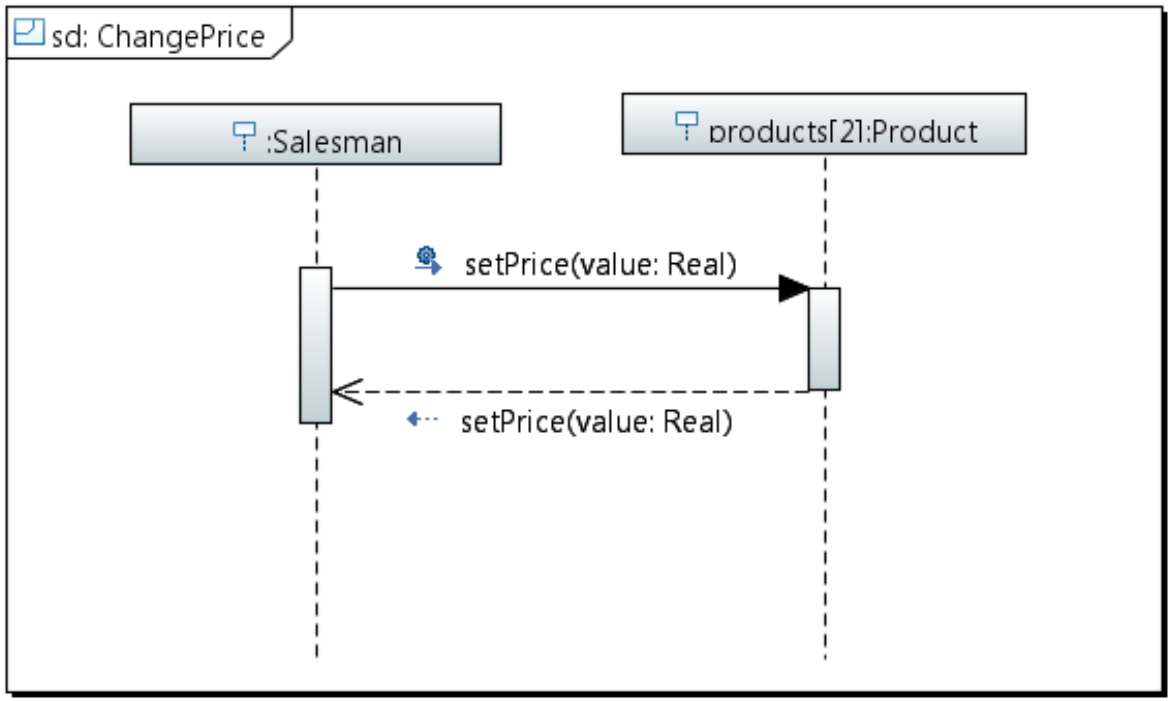
1. **Interaction modeling – sequence diagrams**
   - **Introduction, Messages and Lifelines**
   - **Complex interactions – combined fragment**
2. **Activity diagram**
   - Principles, purpose of UML activities, motivating example
   - Graph like structure and data flow semantics
   - Invocation actions, nodes to coordinate the execution flow

Introduction



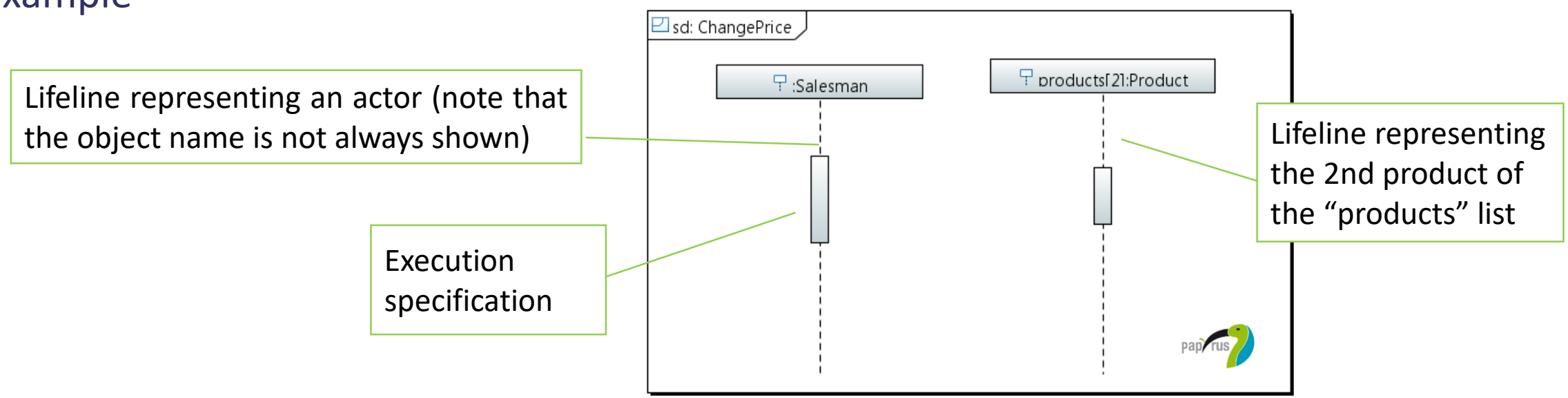


sd = “sequence diagram”



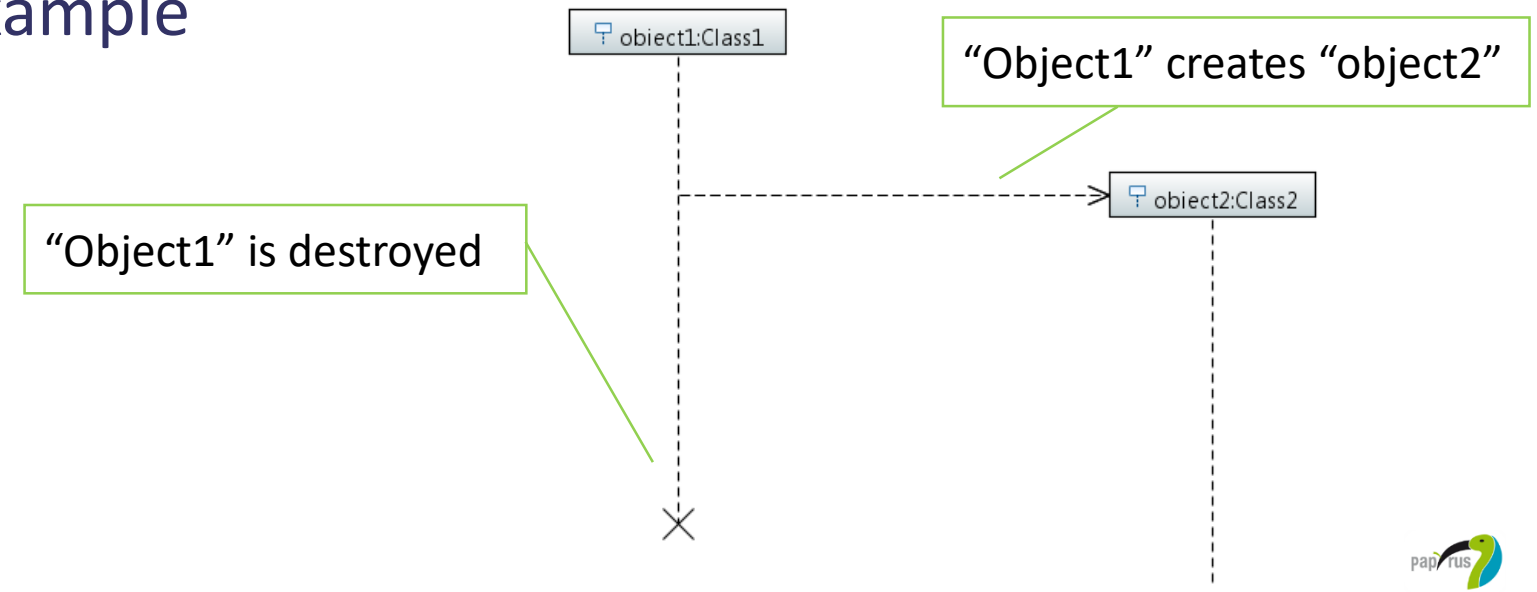
Lifeline – Overview

- Interaction participant, represents an object (modeled with a property) or an actor
- Syntax: `objectName[selector]:typingclassOrActorName`
- The selector is used to choose an object among a collection of objects (e.g. objects[5])
- Lifelines have execution specifications which are period of activities of the object (i.e. it is executing some instructions, or waiting for some event)
- Example



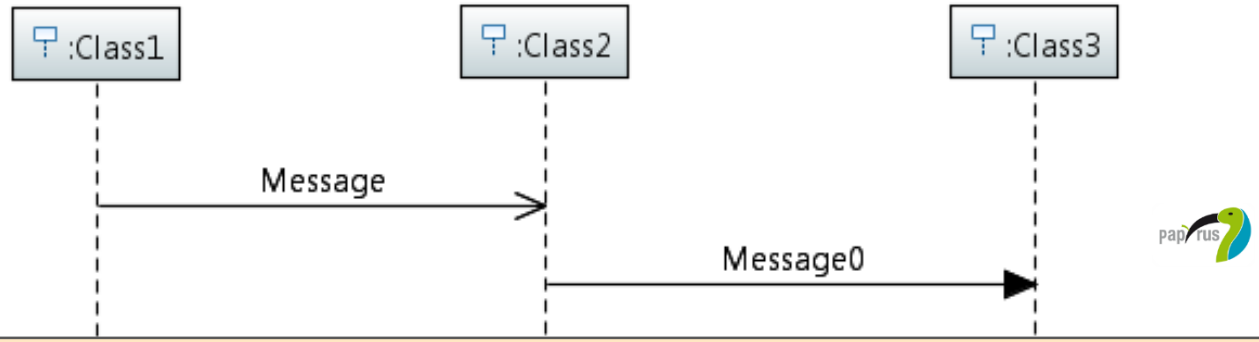
Lifeline – Lifecycle

- An object creation can be represented
- Graphical notation: a dashed arrow pointing to the head of a lifeline
- An object destruction can also be represented
- Graphical notation: a cross at the end of the lifeline
- Example



Message – Overview

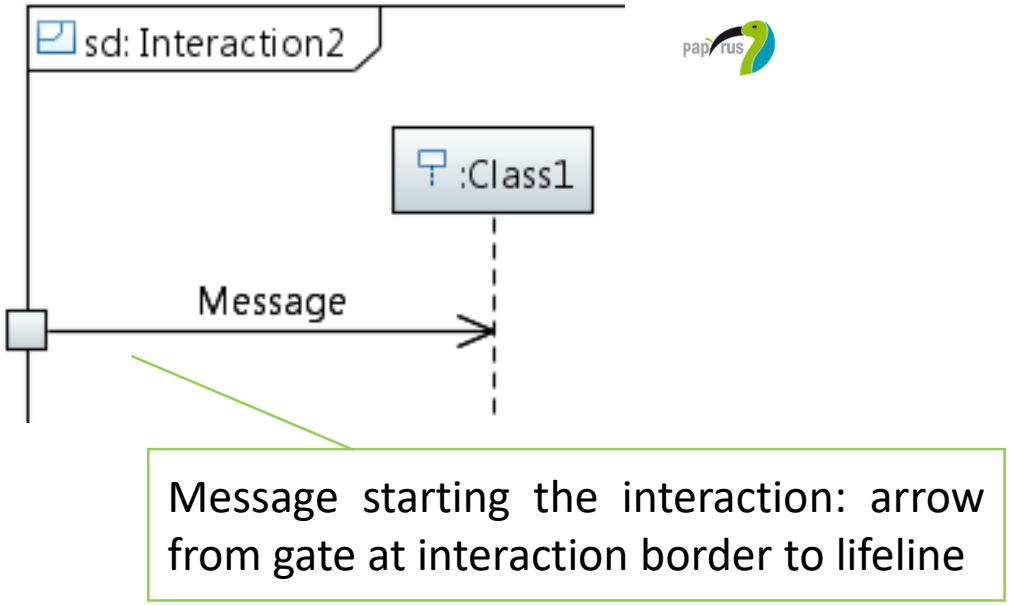
- Represent message exchange in chronological order
- A message defines a particular communication between objects
- Reception of message by a lifeline may trigger an execution specification, showing the handling of the message
- Graphical notation of message: variant of an arrow, depending on the message type (more on this later)



Message exchange between two objects is supposed to begin/end with execution specifications. You´ll find diagrams that don't respect this rule, like this one here. Just remember that **it is incorrect**, even if it's usual!

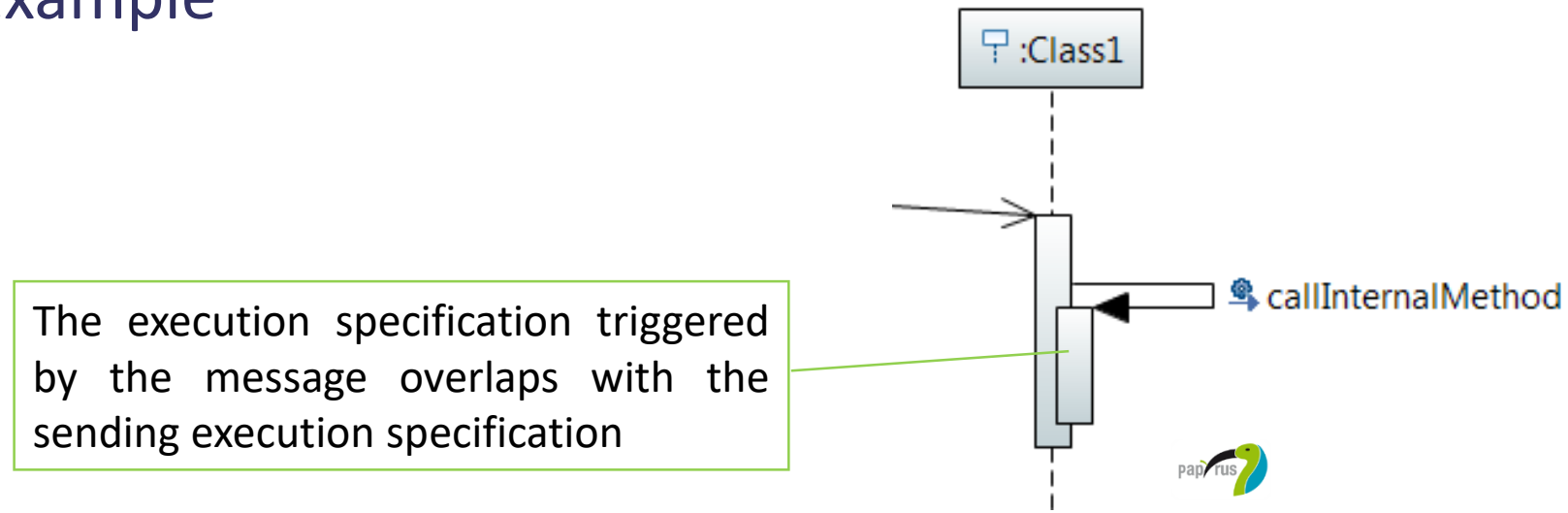
Message – starting an interaction

- A message can be sent from the border of an interaction and received at a lifeline
- Represents the behavior of the system, when a message is received, no matter who the sender is
- Example



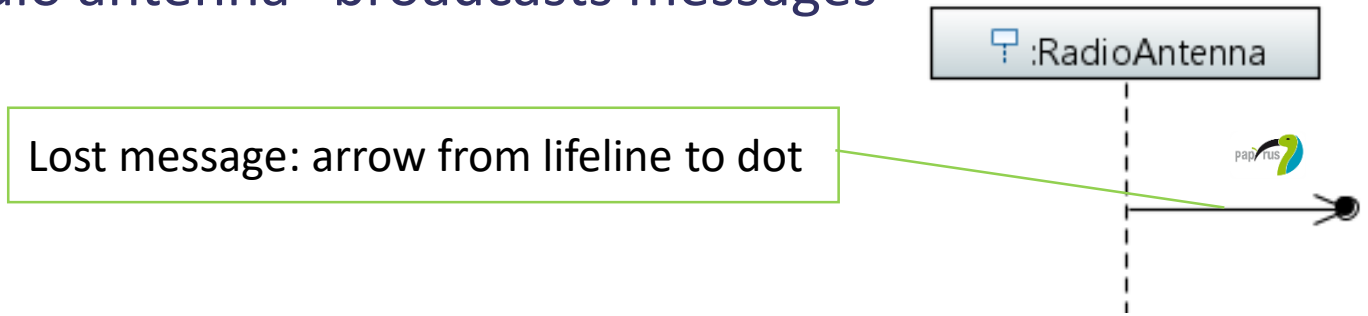
Message sent and received by same lifeline

- A message can be sent from an execution specification on a lifeline, and then received from the same execution specification
- Represents how an object communicates with itself, e.g. by calling one of its own operations
- Example

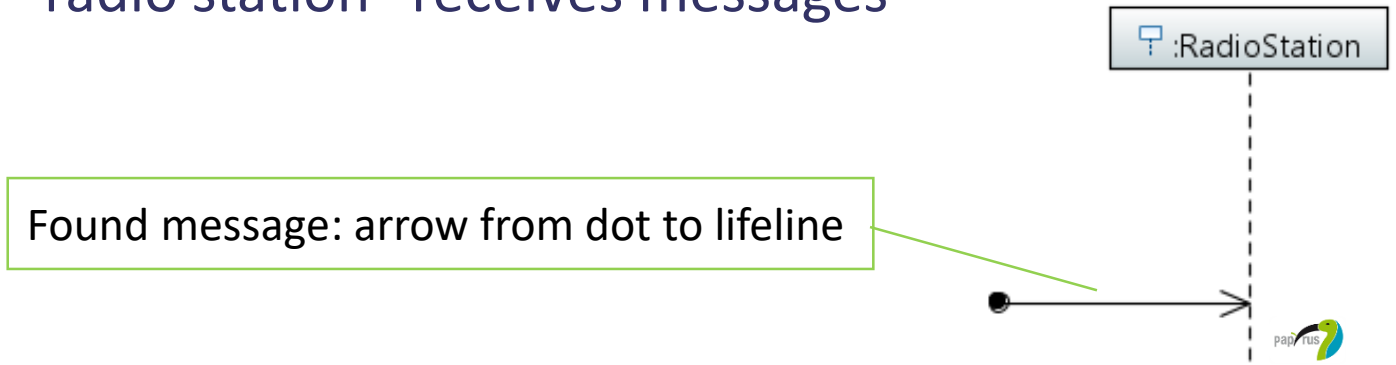


Complete, lost, and found message

- Complete message – send and reception events are known
- Lost message – send event is known, but not reception event, e.g. broadcast
- Example: a “radio antenna” broadcasts messages



- Found message – reception event is known, but not send event
- Example: a “radio station” receives messages



Message – Synchronous

- A synchronous message blocks the sender until an answer from the receiver
- The receiver may send a return message (optional)
- Example:



Sync. message Return message

Synchronous message and operation

- A synchronous message has a signature that may be specified by an operation in a class
- The synchronous message is then an operation call
- The execution specification is then a behavior of the operation
- The return message is the return parameter of the operation

Synchronous Message

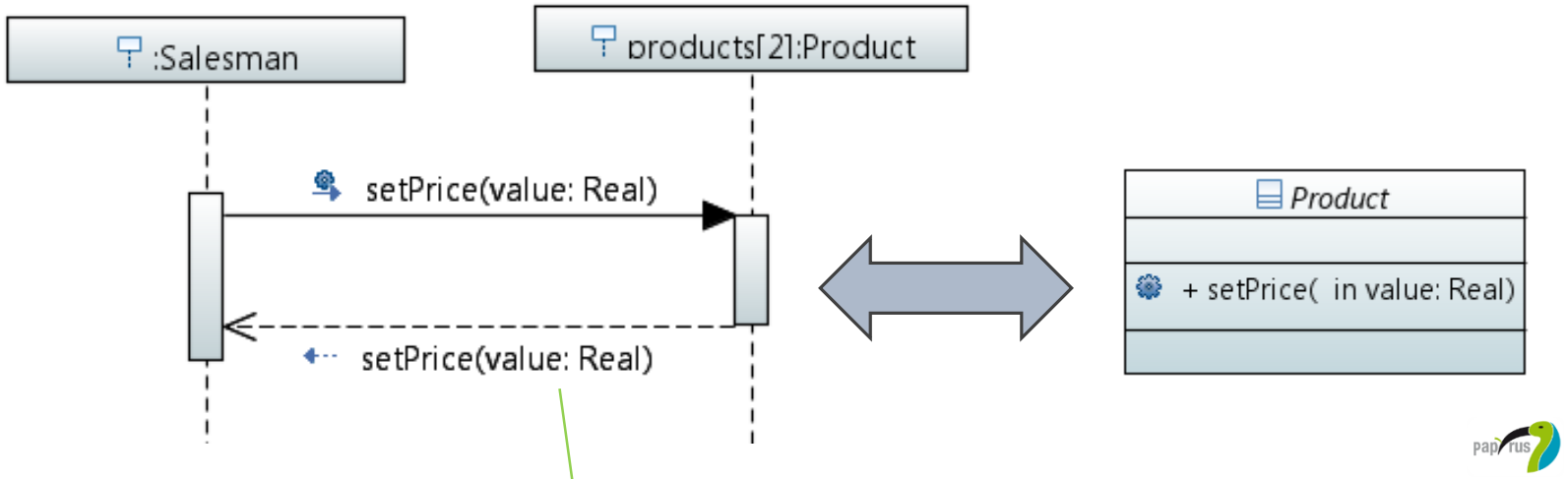
Synchronous message syntax

- General syntax: `opName(args)`
- Arguments syntax: `argName=argVal`
- Example: `setPosition(x=100, y=20)`

Return message syntax

- General syntax: `returnAttr=nameOp(args):returnVal`
- Example: `success=setPosition(x=100, y=20):true`

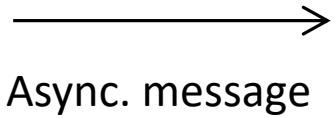
Example – Synchronous and return message



A “salesman” calls an operation of “product”. It is a synchronous message. “Salesman” waits for the answer of “product”, which does not need to contain a return value.

Asynchronous Message – Overview

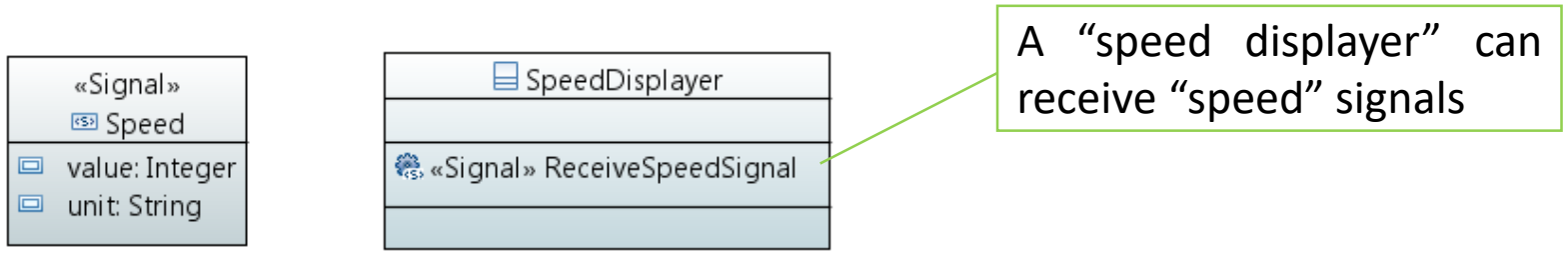
- Non-blocking for sender, i.e. receiver can handle sent message at any time or ignore
- Example



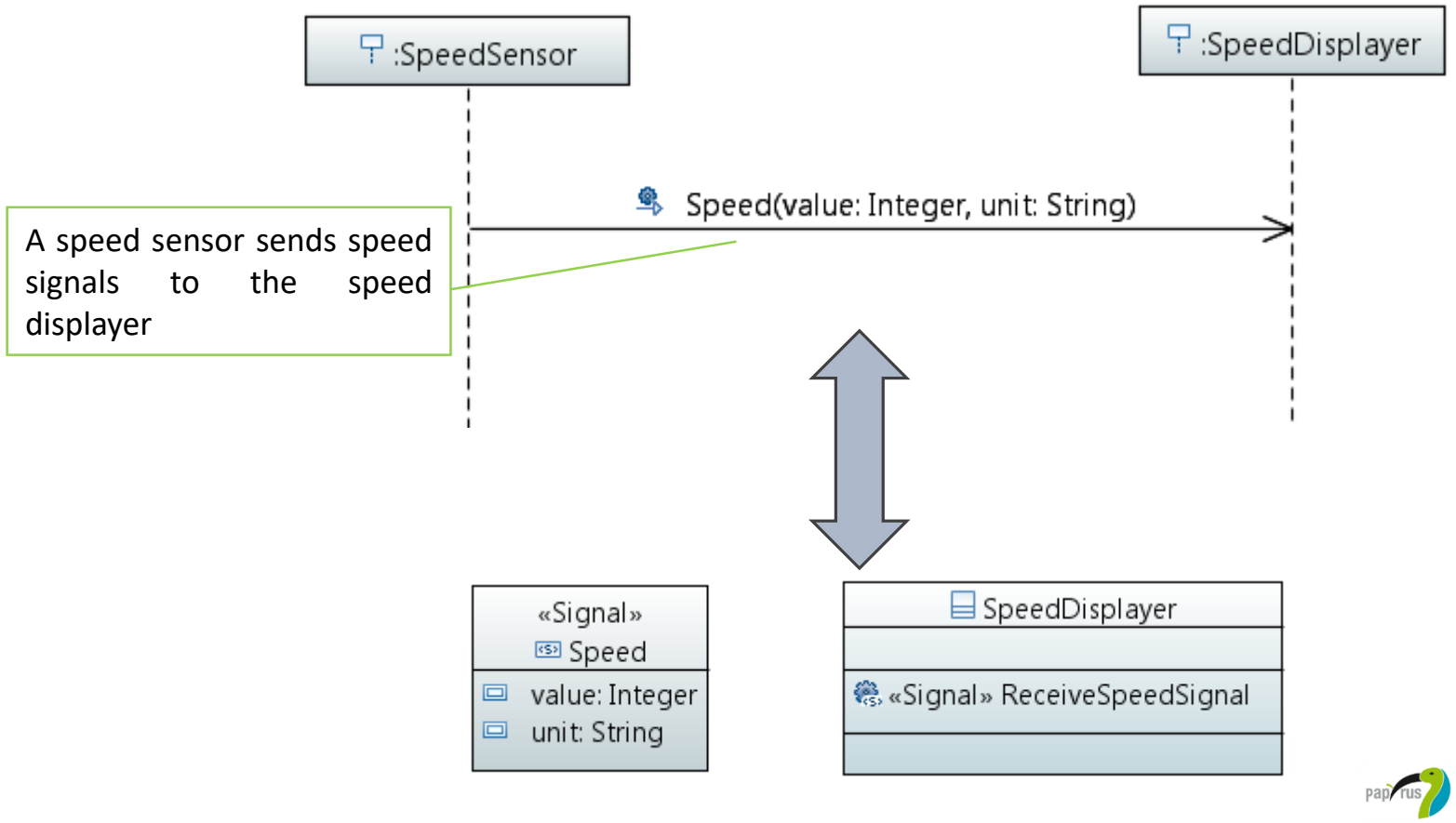
- Syntax is the same as with synchronous messages

Asynchronous message and signals

- Its signature may be specified by an operation or a signal
- Signals (keyword «signal») attributes represent the message parameters
- *Signal reception* indicates that a class can receive and handle a certain signal
- Example



Asynchronous Message – Example



Interactions & component-based modeling

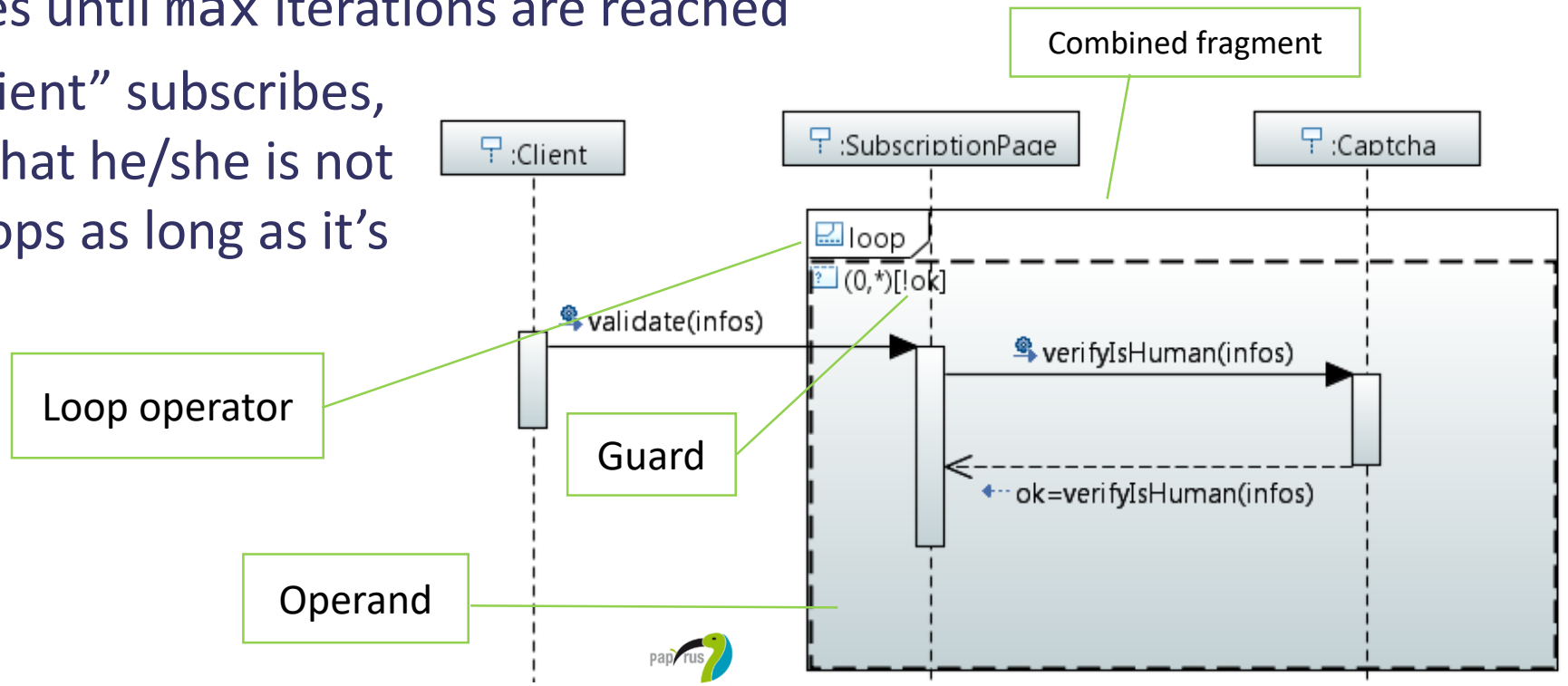
1. **Interaction modeling**
   - **Introduction, Messages and Lifelines**
   - **Complex interations – combined fragment**
2. **Activity diagram**
   - Principles, purpose of UML activities, motiving example
   - Graph like structure and data flow semantics
   - Invocation actions, nodes to coordinate the execution flow

Combined Fragment – Overview

- Brings control flow to interaction (i.e. different and conditional execution paths)
- Is divided into one or several pieces of interaction called interaction operands, e.g. may have two operands “if (…)”, and “else” to represent conditional execution
- Interaction operands are controlled by the **interaction operator** of the combined fragment:
  - Choices and loops: `loop, alternative, option, break`
  - Parallelization: `parallel, critical region`
  - Message sending: `negative, ignore, consider, assertion`
  - Message ordering: `weak sequencing, strict sequencing`
- Operand may have a guard: an interaction constraint that controls the execution of the piece of interaction represented by the operand

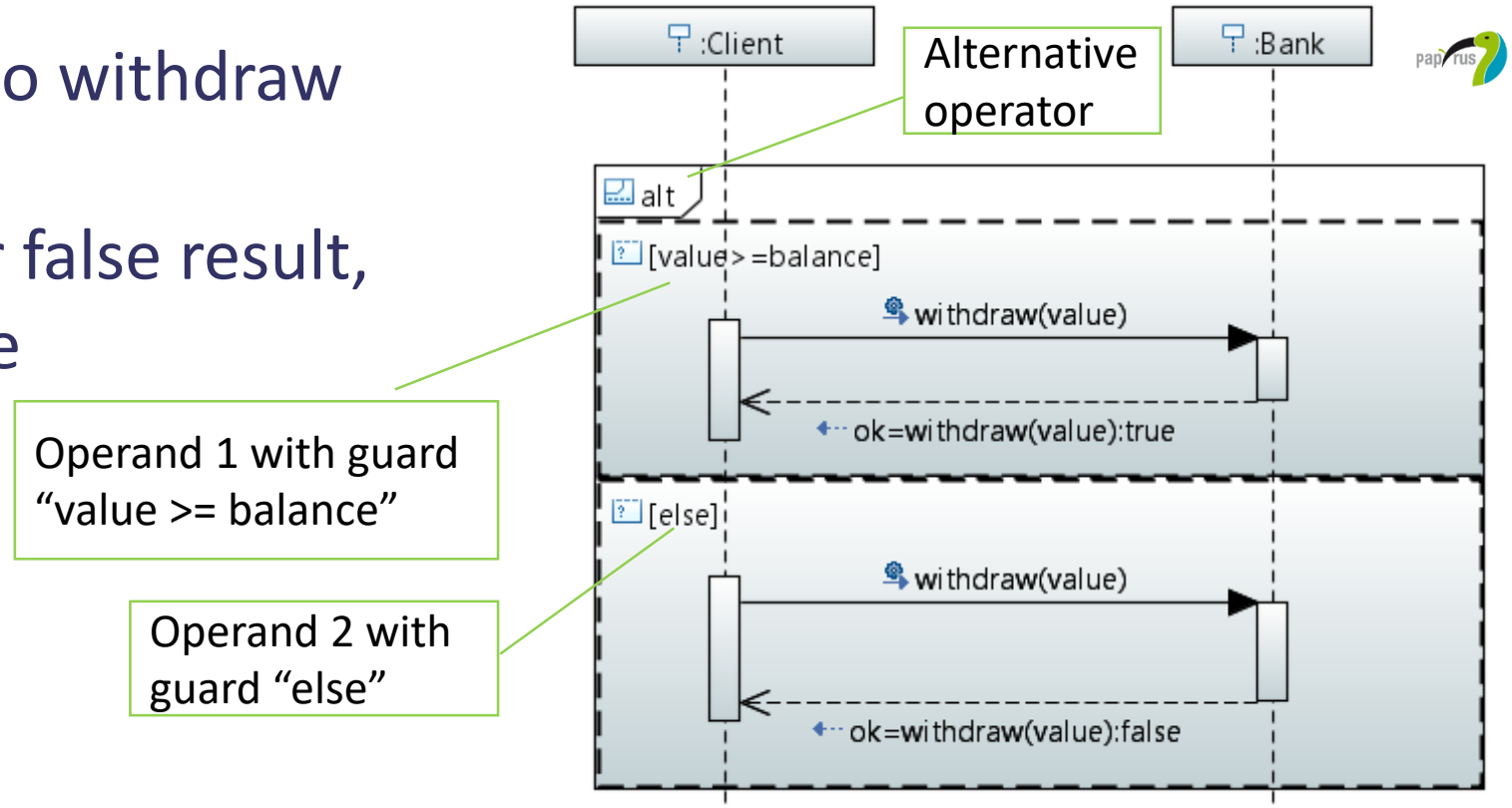
Combined Fragment – Loop operator

- Has one operand with a guard: `(min, max)[condition]`
- Alternative syntax for the guard: `(n, n)` is noted `(n)`
- Loop is repeated at least `min` times before a `condition` is tested
- If true, loop continues until `max` iterations are reached
- Example: When a “client” subscribes, a “captcha” verifies that he/she is not a bot; verification loops as long as it’s not ok



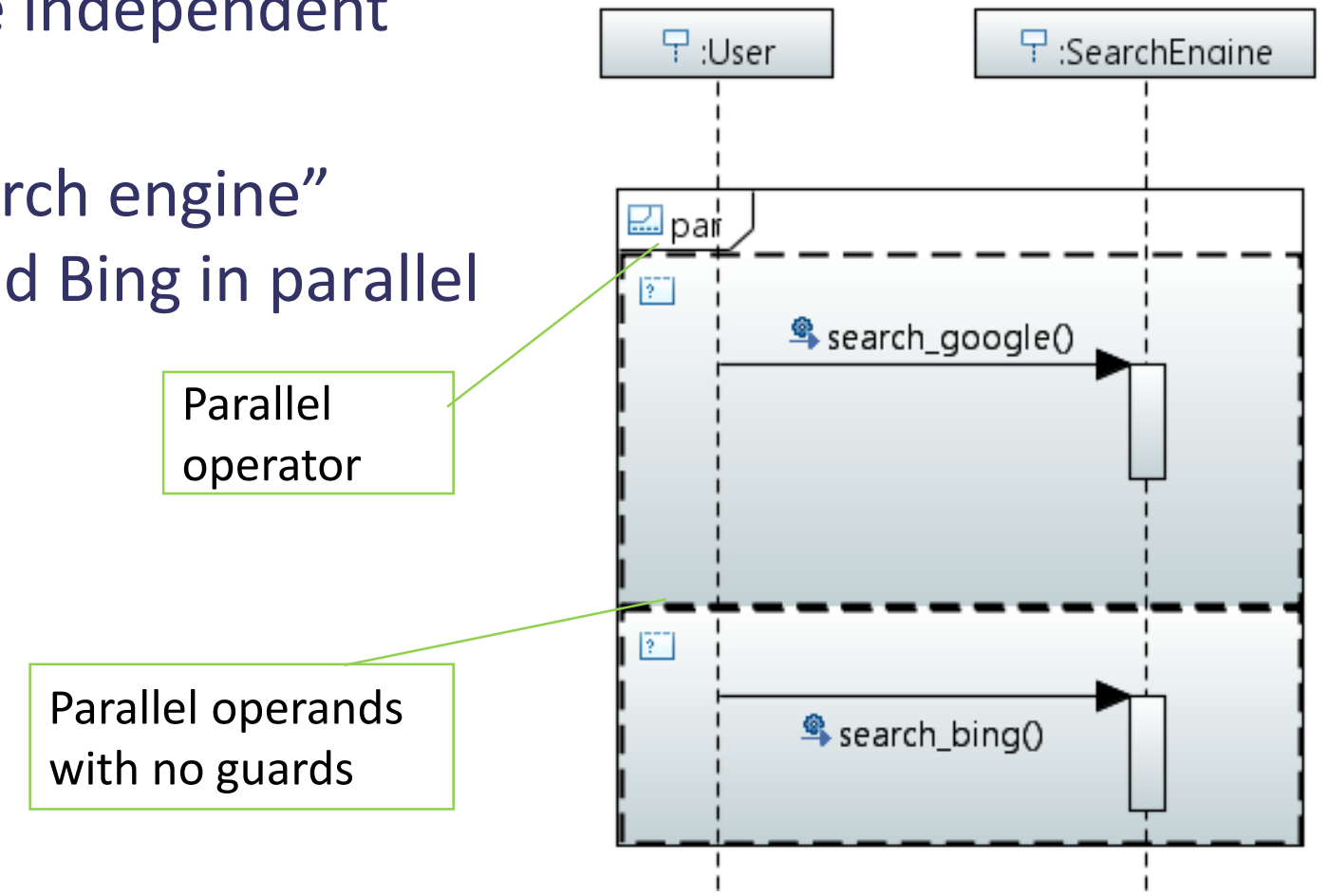
Combined Fragment – Alternative operator

- Specifies a choice of alternative interactions
- Each operand is an alternative interaction and has a guard (in [..]) controling if the interaction is executed.
- Example: “Client” tries to withdraw money from “bank”.
  “Bank” returns a true or false result, according to the balance



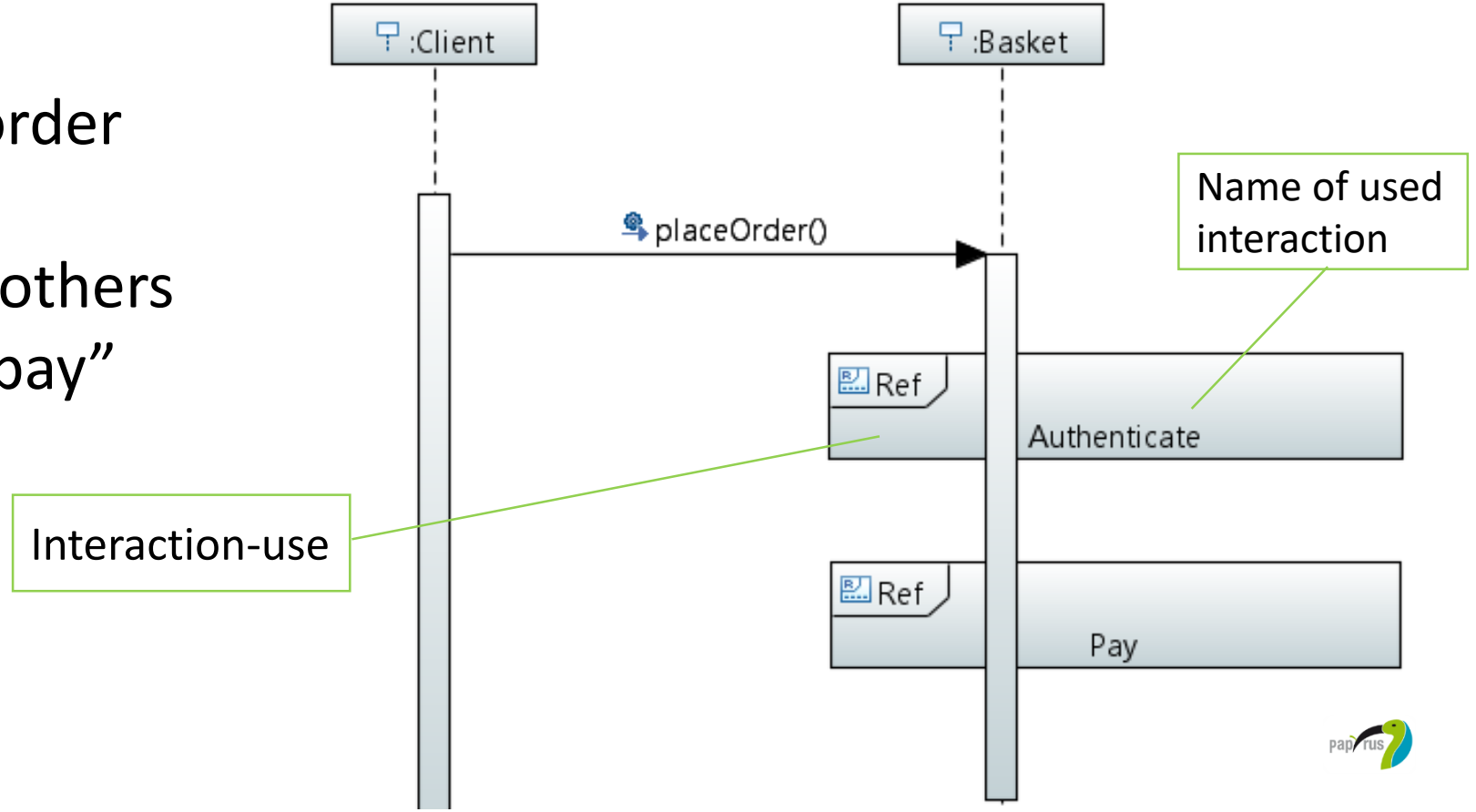
Combined Fragment – Parallel operator

- Specifies a parallel interactions
- Parallel interactions are independent
- Example
  A “user” can use a “search engine” searching on Google and Bing in parallel



Interaction reuse

- An interaction can be re-used within another interaction through the interaction-use fragment
- Example
  A “client” places an order from the “basket”;
  Interaction uses two others “authenticate” and “pay”



Summary

- Interaction models describe *possible* execution traces and scenarios of a system (not the exhaustive behavior of a system)
- A lifeline is a participant of an interaction, it represents an object or an actor, has execution specifications that represent periods of activities of the participant
- Objects represented by lifelines exchange messages in chronological order; reception of a message triggers an execution specification
- Synchronous messages block sender until receiver answers; are usually specified by an operation; receiver may optionally send return message
- Asynchronous messages don’t block the sender, are specified by signal or operation
- A combined fragment can be used to represent several parts of an interaction, whose executions are controlled (brings control flow)
- An interaction can be re-used with the interaction use fragment

Behavior diagrams – interactions & activities

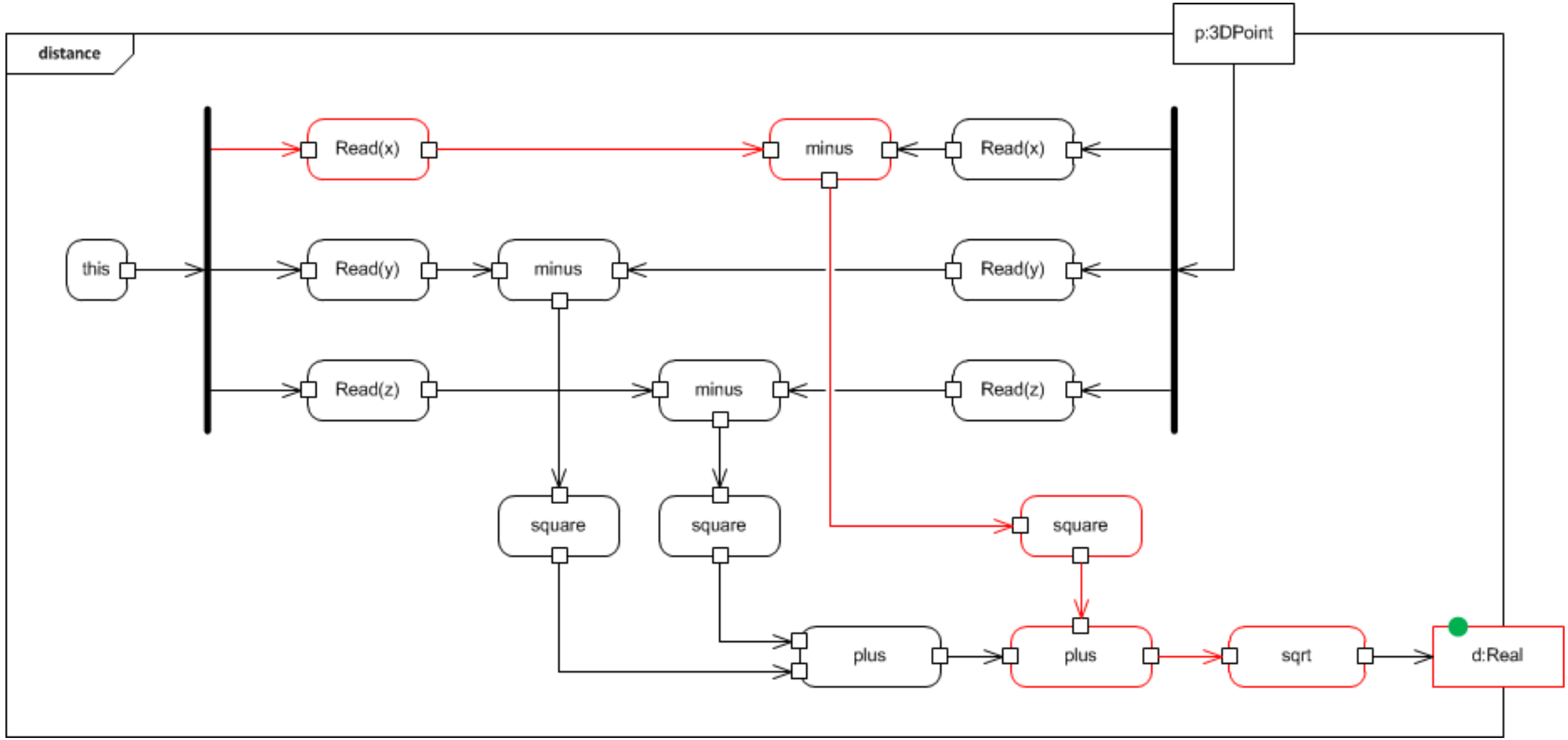
1. Interaction modeling – sequence diagrams
   - Introduction, Messages and Lifelines
   - Complex interations – combined fragment
2. **Activity diagram**
   - **Principles, purpose of UML activities, motivating example**
   - Graph like structure and data flow semantics
   - Nodes to coordinate the execution flow

What is the purpose of activities?

Based on UML 2.5

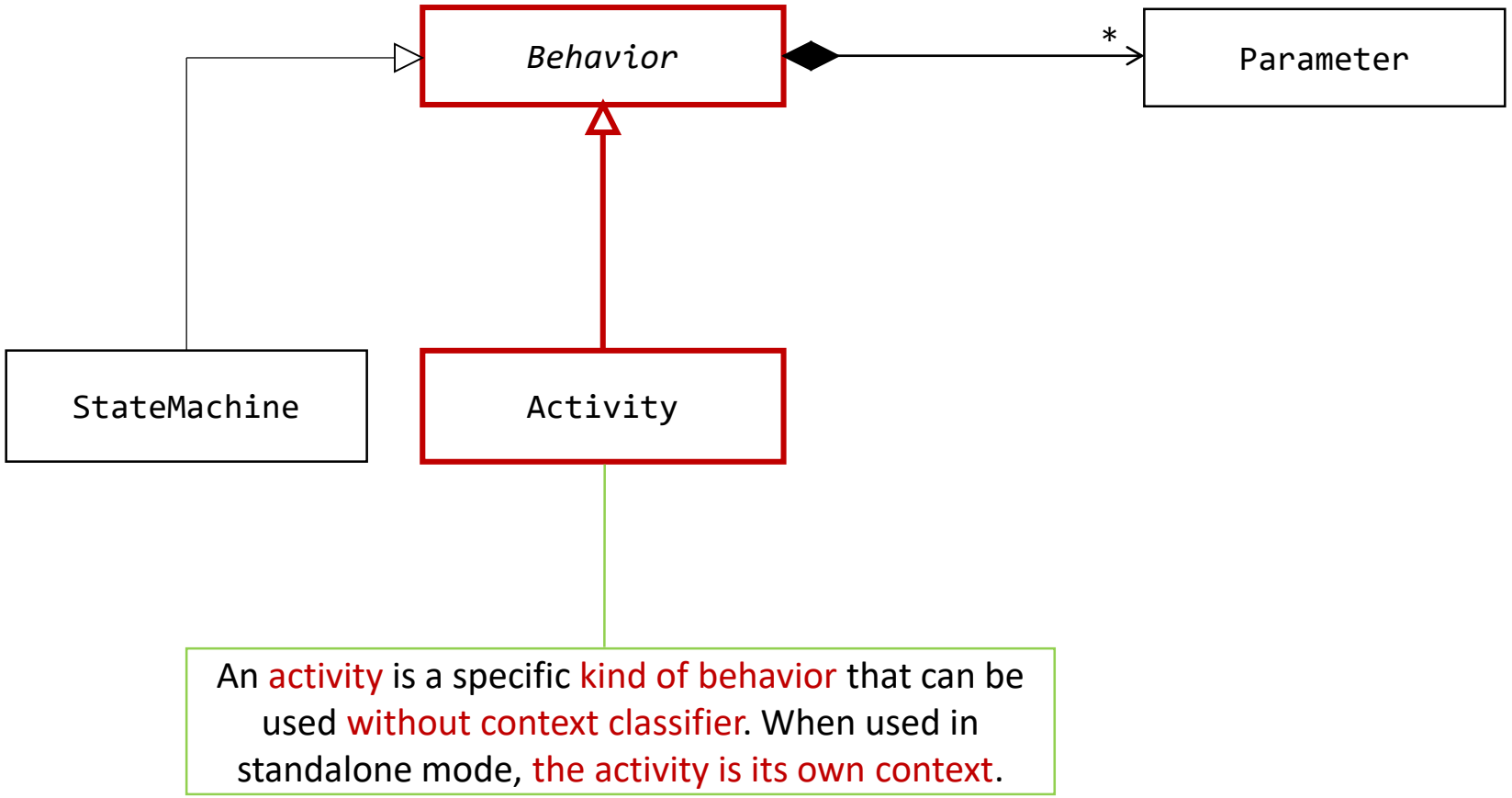
- “Activities may describe procedural computation…”
- “…in an object-oriented model, they may be invoked indirectly as methods bound to Operations that are directly invoked…”.
- “Activities can also be used for information system modeling to specify system level processes…”

Activity diagram example – 3d distance



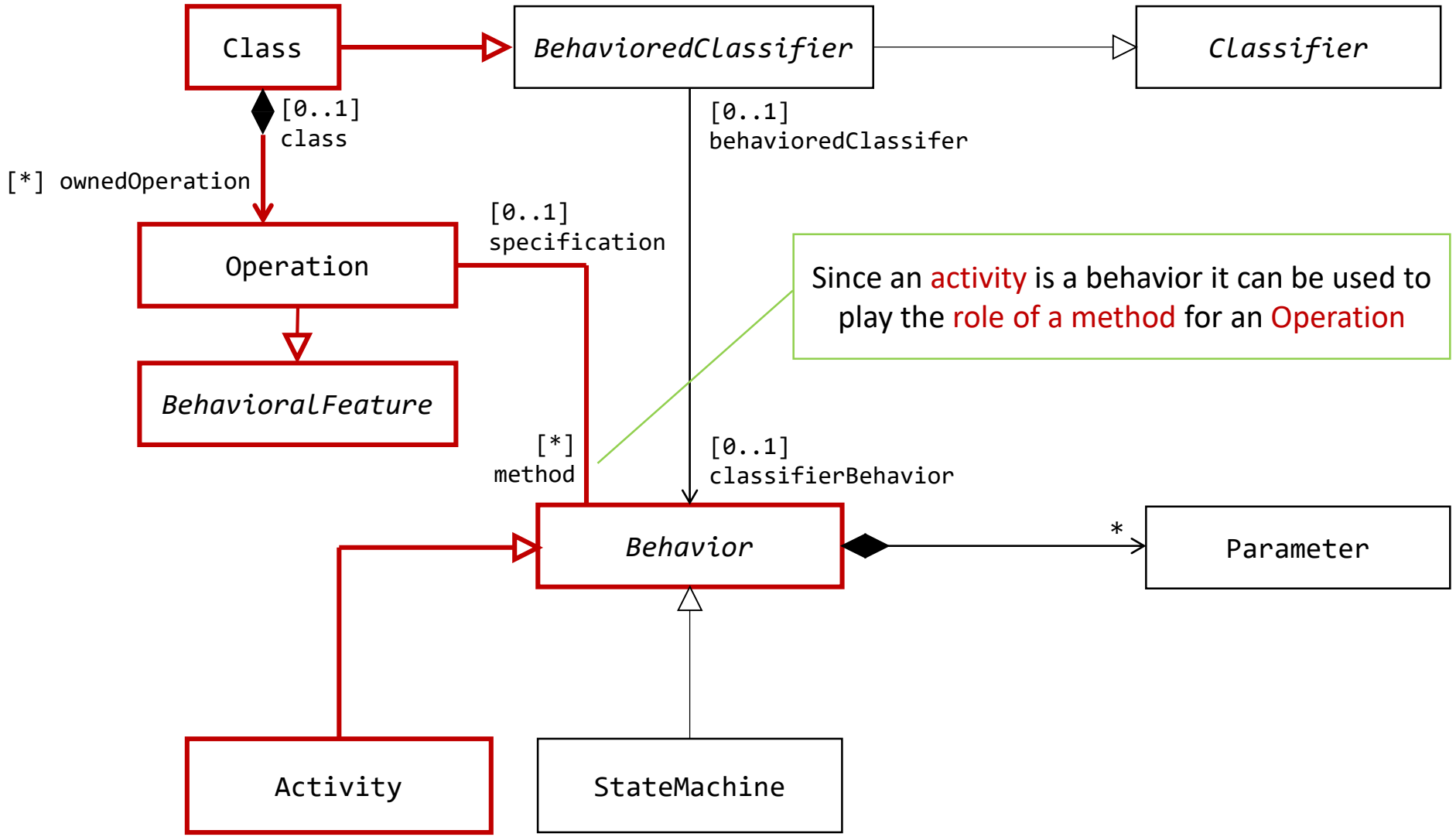
Activity usages allowed by UML (1/3)

In standalone mode



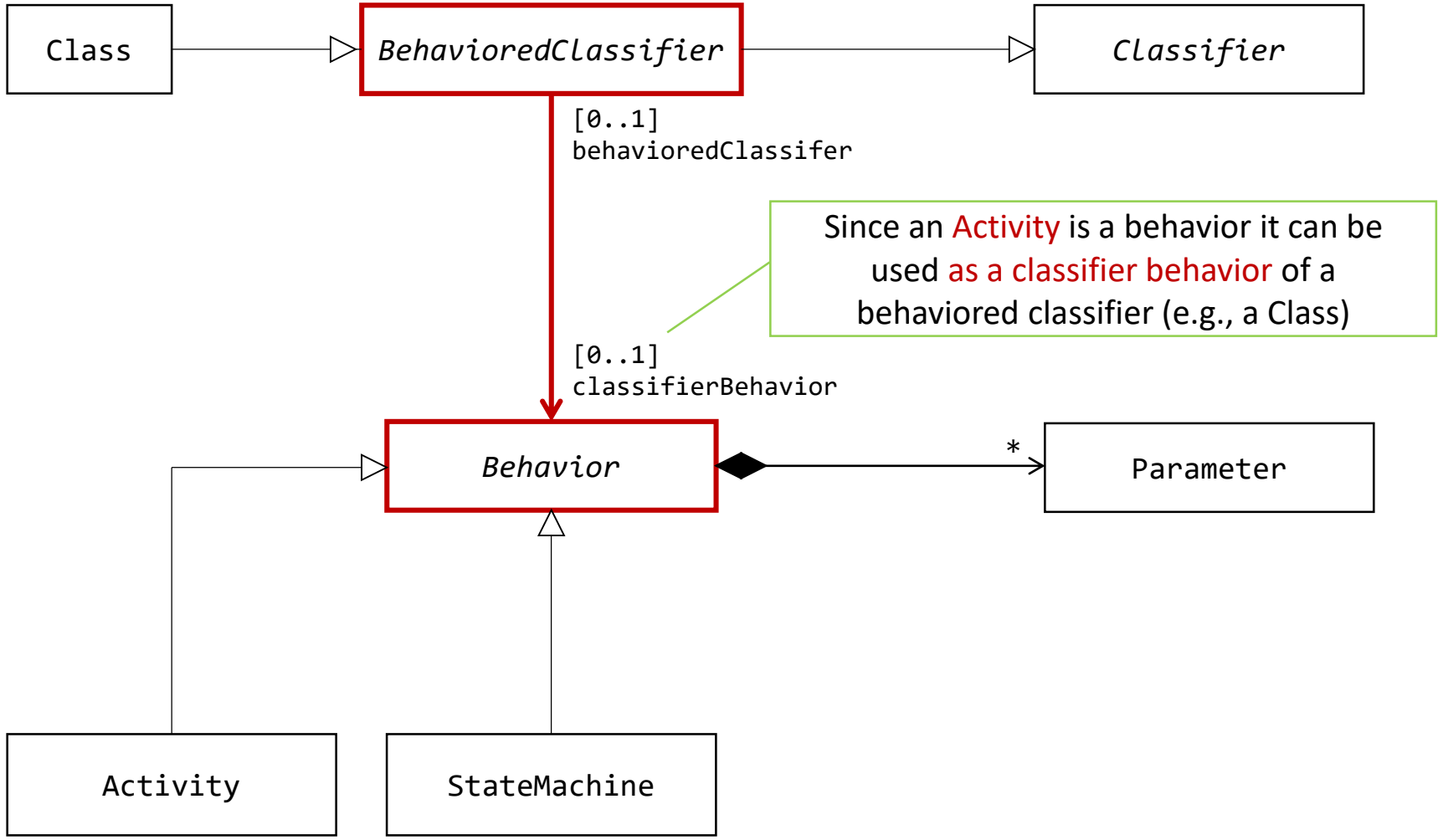
Activity usages allowed by UML(2/3)

As an operation implementation



Activity usages allowed by UML (3/3)

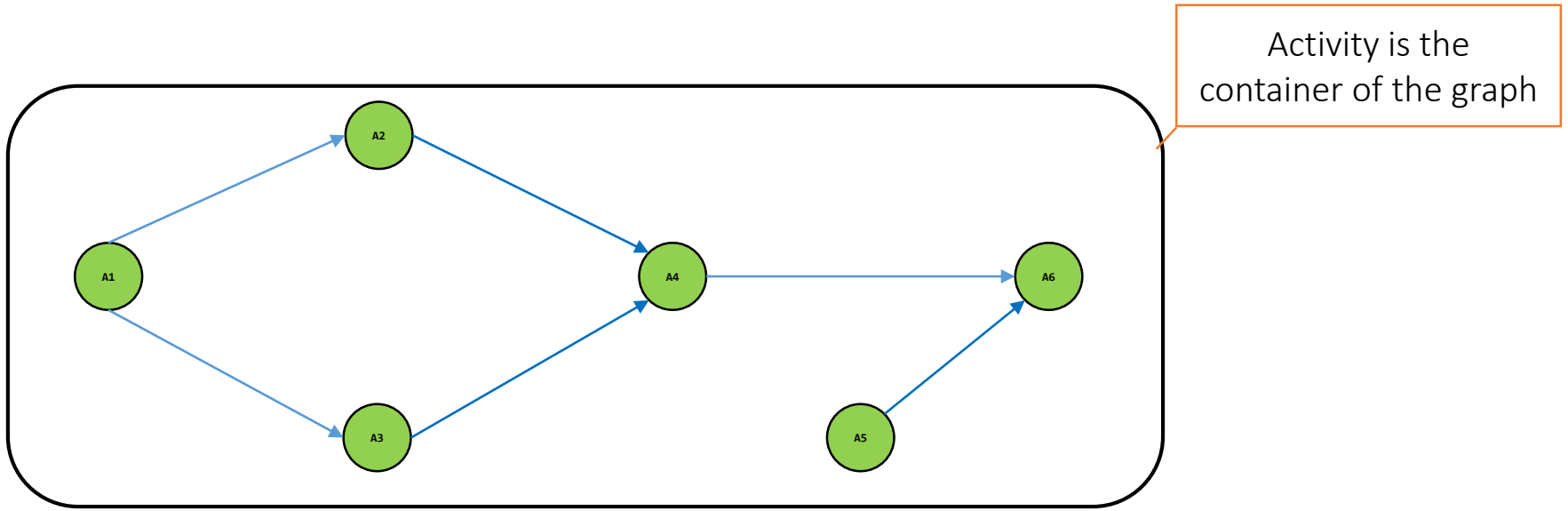
As classifier behavior



Behavior diagrams – interactions & activities

1. Interaction modeling – sequence diagrams
   - Introduction, Messages and Lifelines
   - Complex interactions – combined fragment
2. **Activity diagram**
   - Principles, purpose of UML activities, motivating example
   - **Graph like structure and data flow semantics**
   - Nodes to coordinate the execution flow

Directed Graph like structure (1/2)

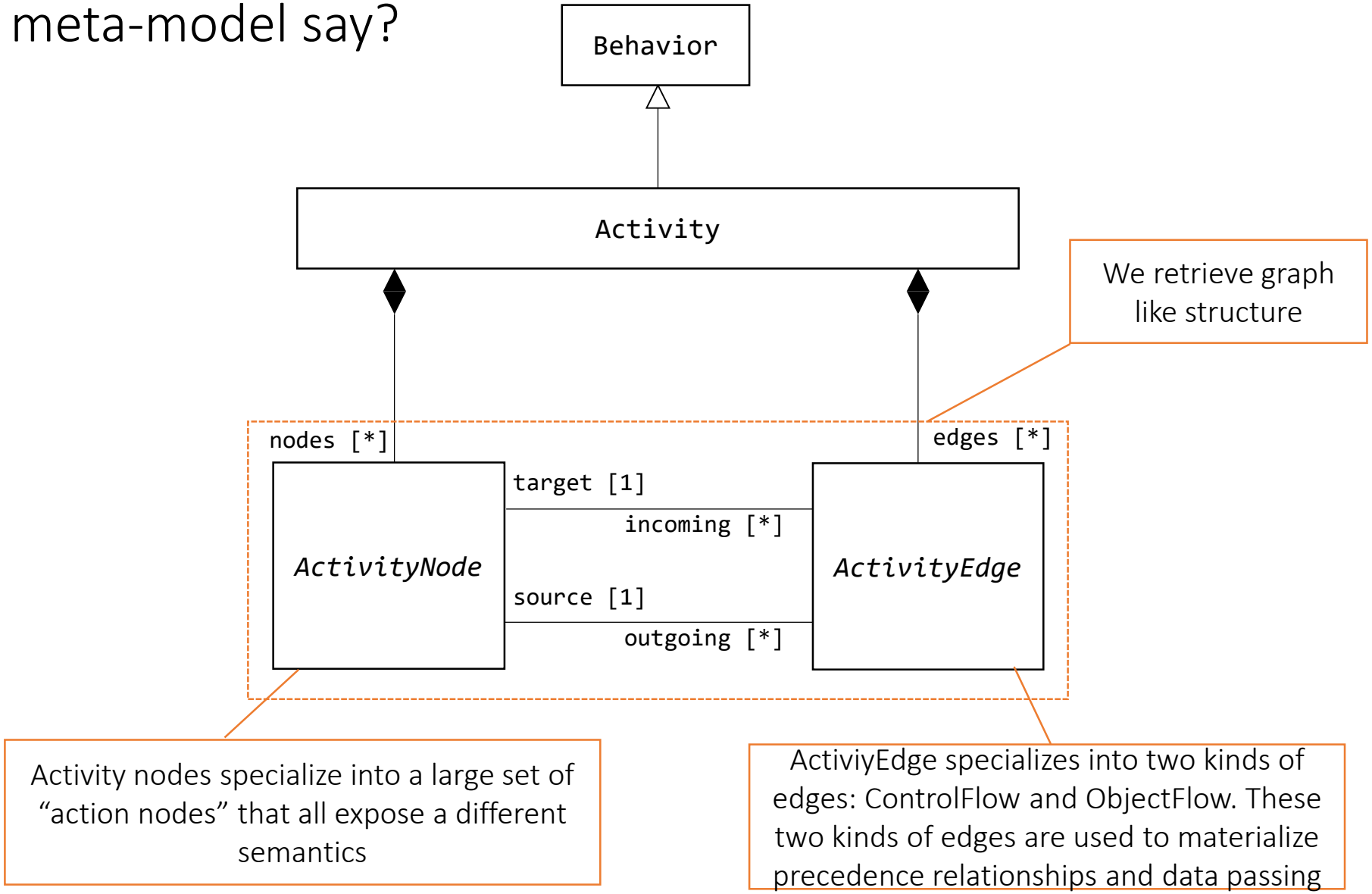


Graph composition

- The activity itself is the container of the graph
- The nodes denotes actions to realize
- The edges denote precedence relationships between action nodes
  - Edges in red denote only precedence relationships
- The edges denotes flowing of data between action nodes
  - Edges in blue denote a precedence relationship as well as data flowing from an action to another

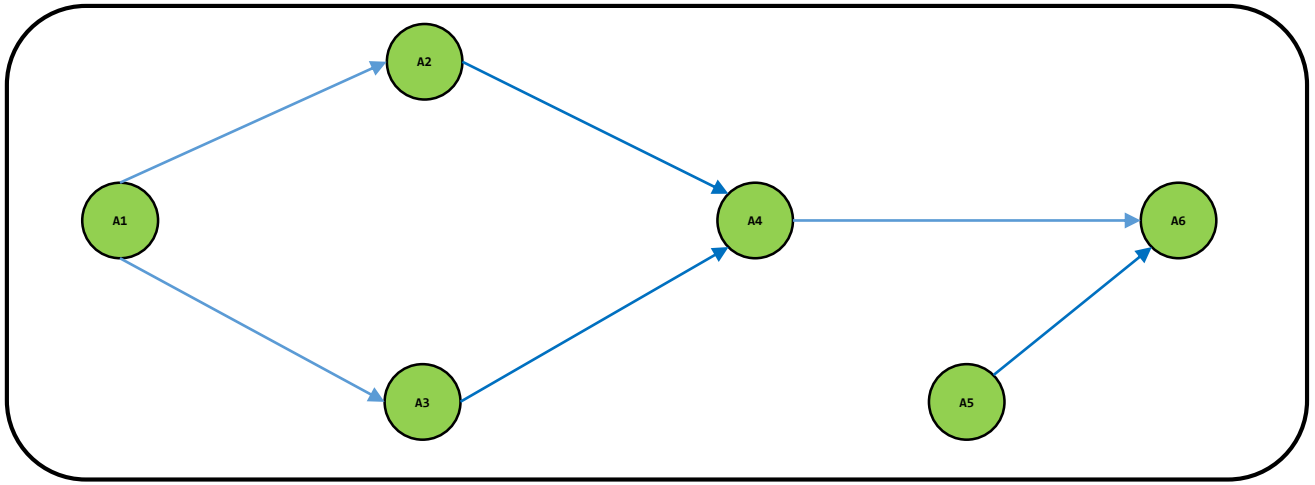
Directed Graph like structure (2/2)

What does the meta-model say?



Data-flow semantics (1/4)

The execution rules

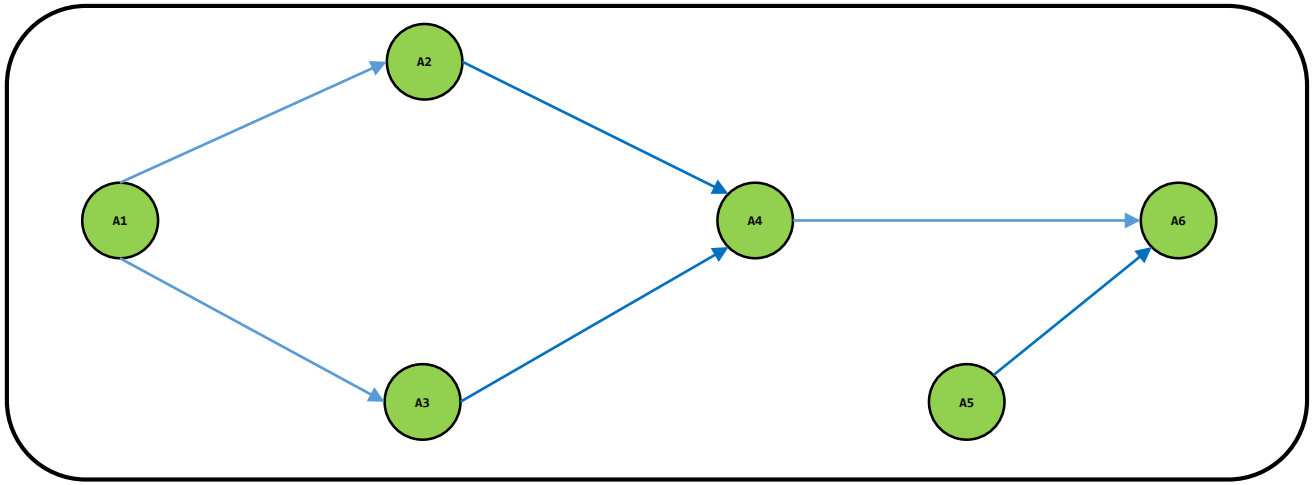


- A node can be executed when
  - Its control and data dependencies are satisfied
  - Example: A4 cannot be executed before A1-A2-A3 are done

- When the activity execution starts
  - Nodes that have no dependencies to satisfy are executed concurrently
  - Example: A1 and A5 executes concurrently

Data-flow semantics (2/4)

Partial execution orders

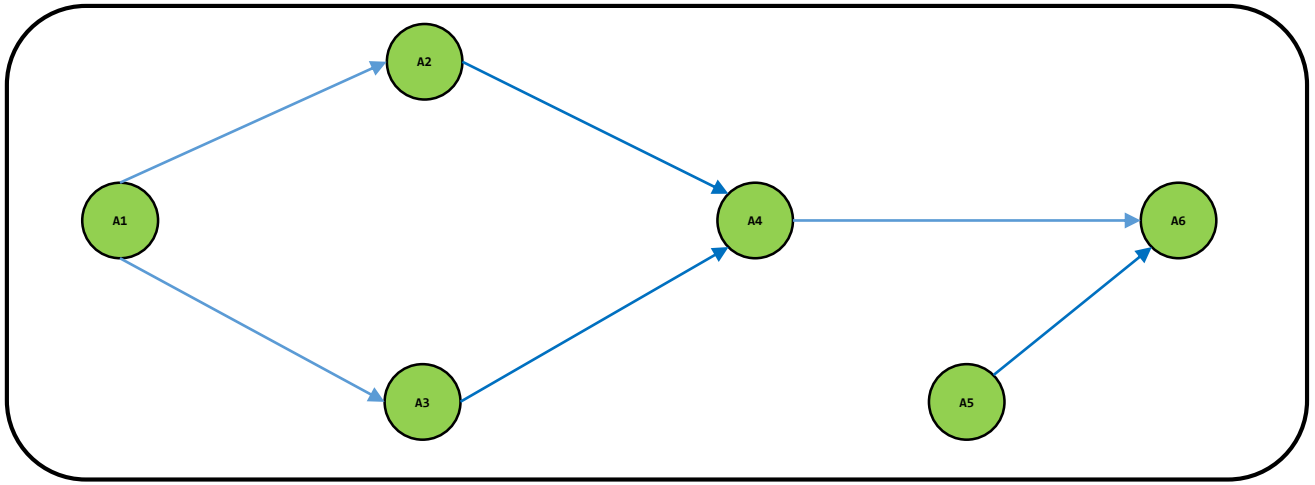


We can be sure that	However we cannot say that	Partial execution order:
A2 after A1	A5 after A1 (and reverse)	A1 and A5 (parallel)
A3 after A1	A3 after A2 (and reverse)	A2 and A3 (parallel or sequence)
A4 after A2, A3		A4
A6 after A4, A5		A6

Dependent on the model form (i.e., is parallelism explicitly described with a Fork ?)

Data-flow semantics (3/4)

How can we force a particular order?



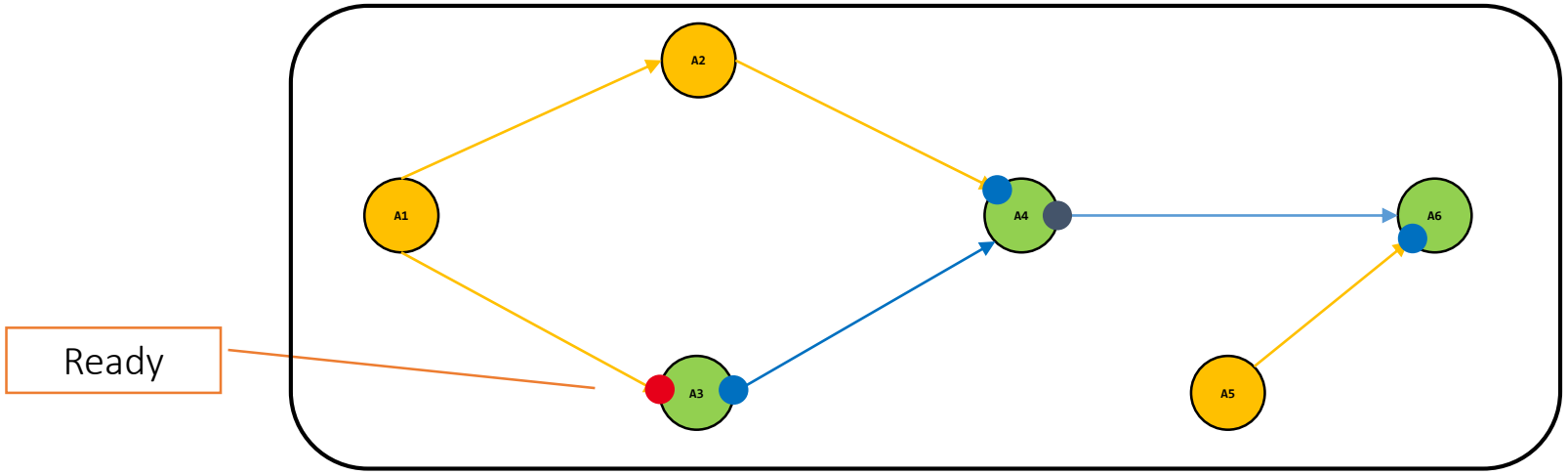
Question:

- How can we say that A2 will be executed each time after A3 ?

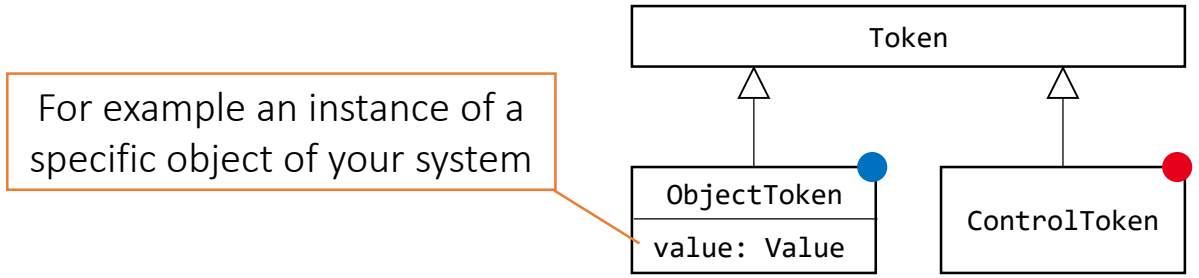


Data-flow semantics (4/4)

Execution propagation: token based system



- Materialized by tokens flowing through edges
  - Control token: propagate across control flows
  - Object token: propagate across the object flows and ships data



Activity parameters nodes



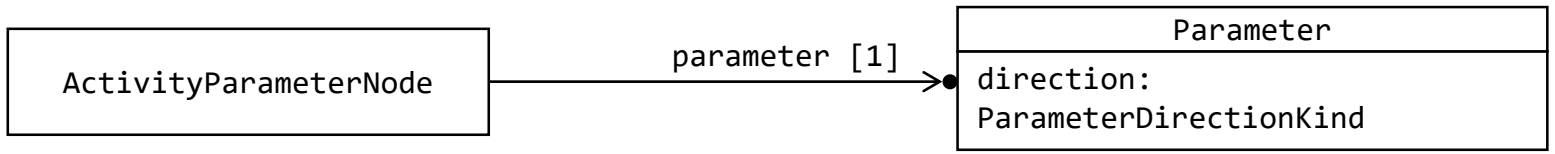
Activity parameters nodes

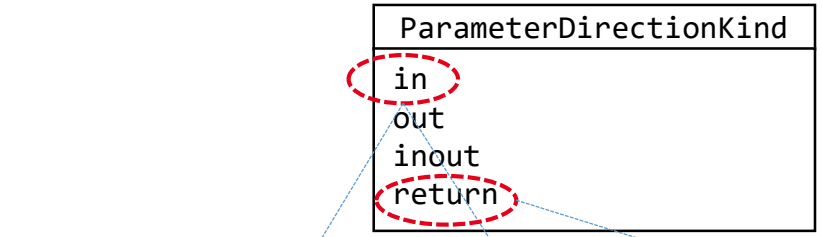
- Denote points used to provide an activity with input values
- Denote points used by an activity to provide output values
- They are typed (not mandatory)
  - Can be derived from the parameter attached to the activity parameter node

```
activity main() {
  v = f(5,2)
}
```

A valid call for this activity

Activity parameters nodes – direction





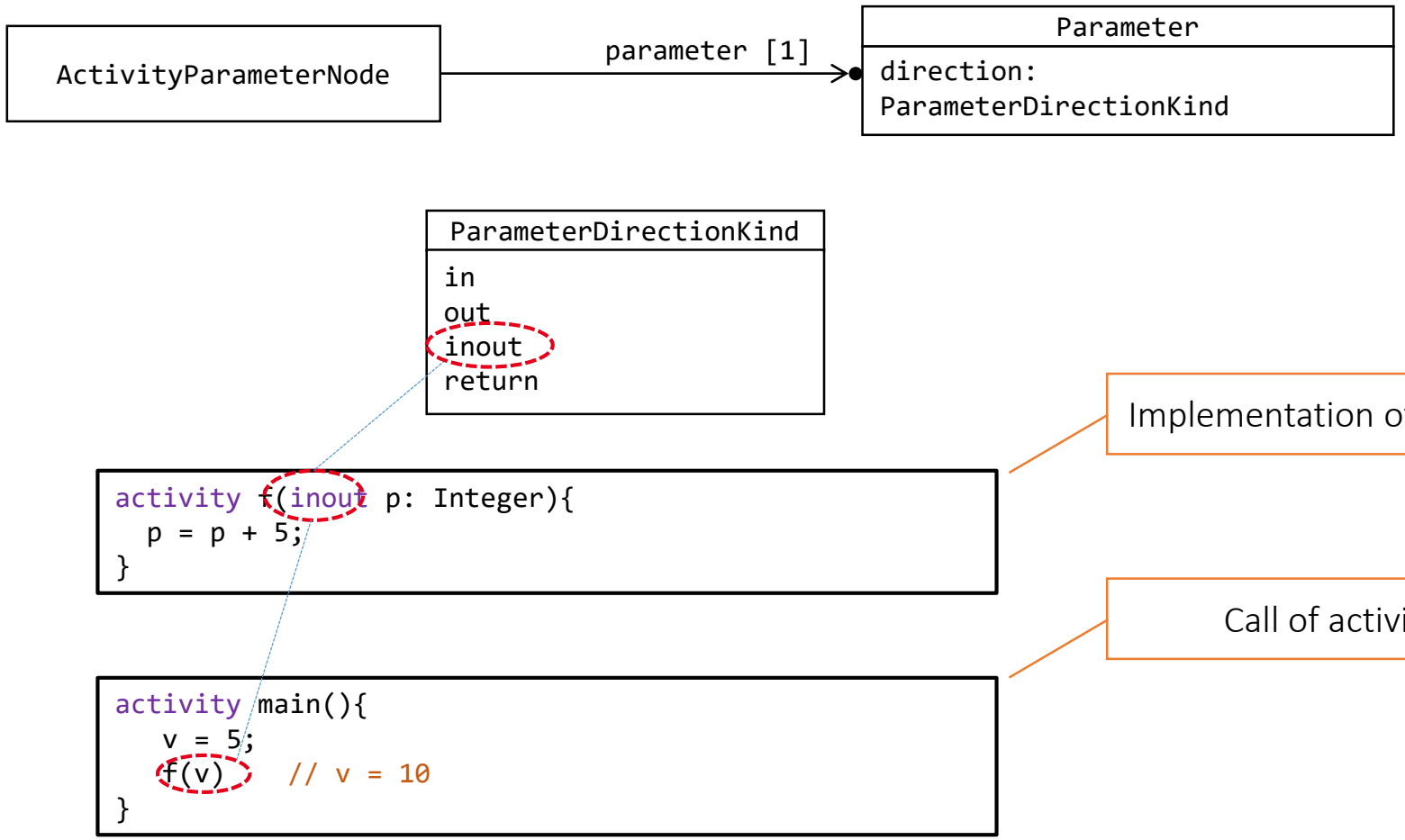

```
activity f(in p1: Integer, in p2: Integer) : Integer{
  return p1 + p2;
}
```

Implementation of activity “f”

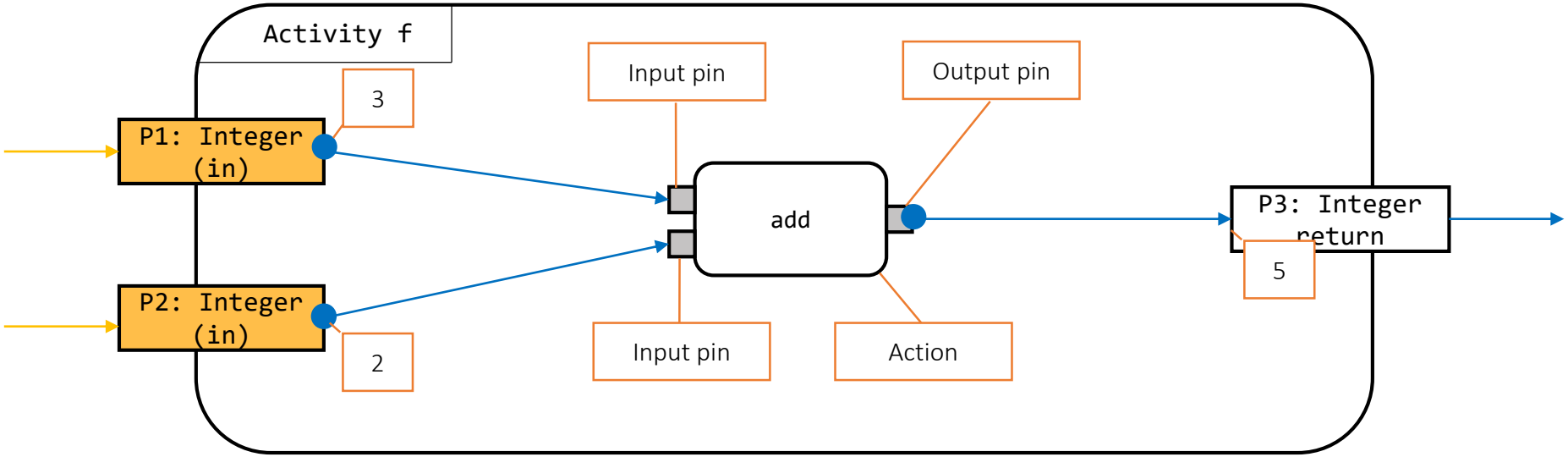
```
activity main(){
   let a:Integer = 3;
   let b:Integer = 2;
  v = f(a, b)     // v = 5
}
```

Call of activity “f”

Activity parameters nodes – direction



Actions – Input and output pins



Actions

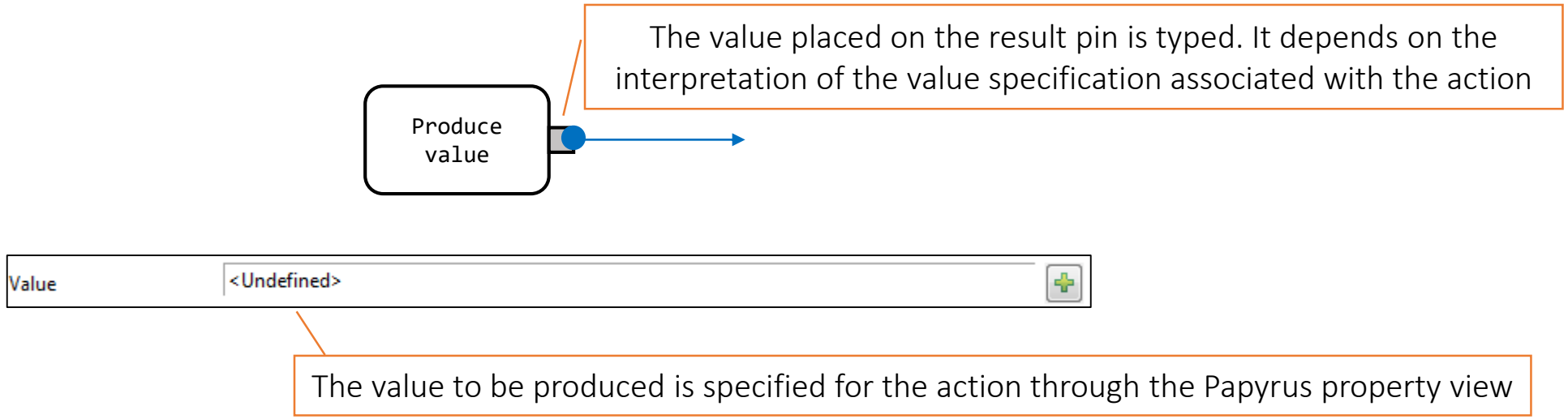
- Can consume and/or produced data
  - Data consumed by an action come from its input pin(s)
  - Data produced are placed on its output pin(s)

- Note: it is not mandatory for an action to consume or produce data

Behavior diagrams – interactions & activities

1. Interaction modeling – sequence diagrams
   - Introduction, Messages and Lifelines
   - Complex interactions – combined fragment
2. **Activity diagram**
   - Principles, purpose of UML activities, motivating example
   - Graph like structure and data flow semantics, Base actions
   - **Invocation actions, nodes to coordinate the execution flow**

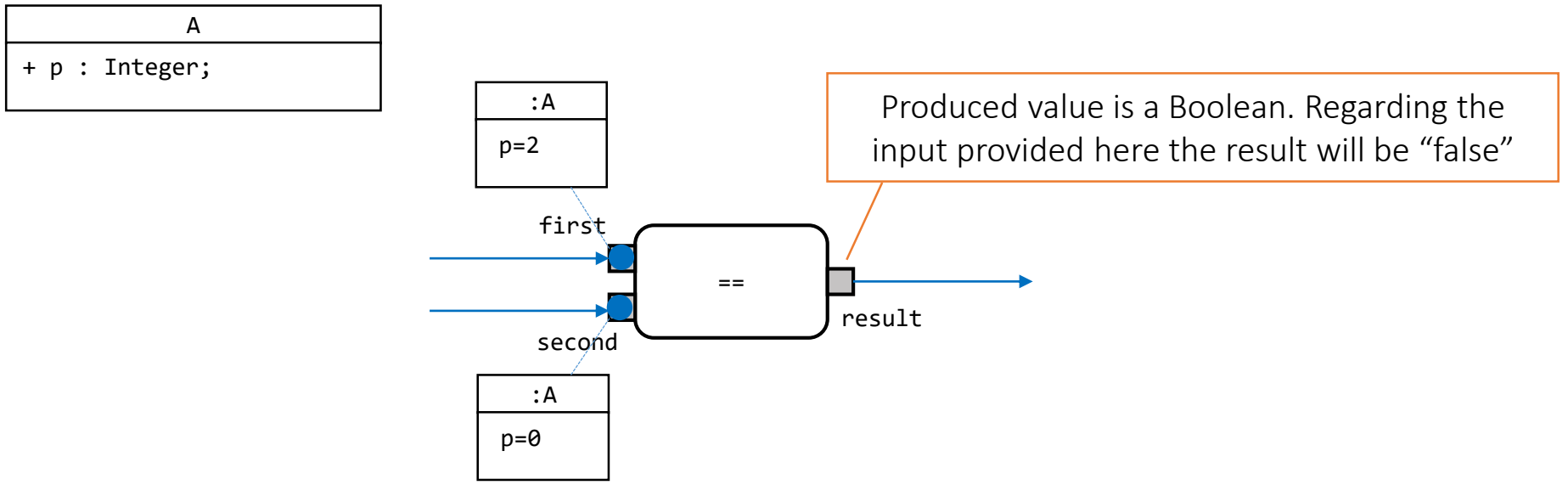
Base action – Value specification



ValueSpecificationAction

- Produce a value on its result output pin from any kind of value specification
  - Expression (e.g. StringExpression, TimeExpression)
  - Literal value (e.g. LiteralInteger)
  - Duration
  - InstanceValue (e.g. an instance of a class)

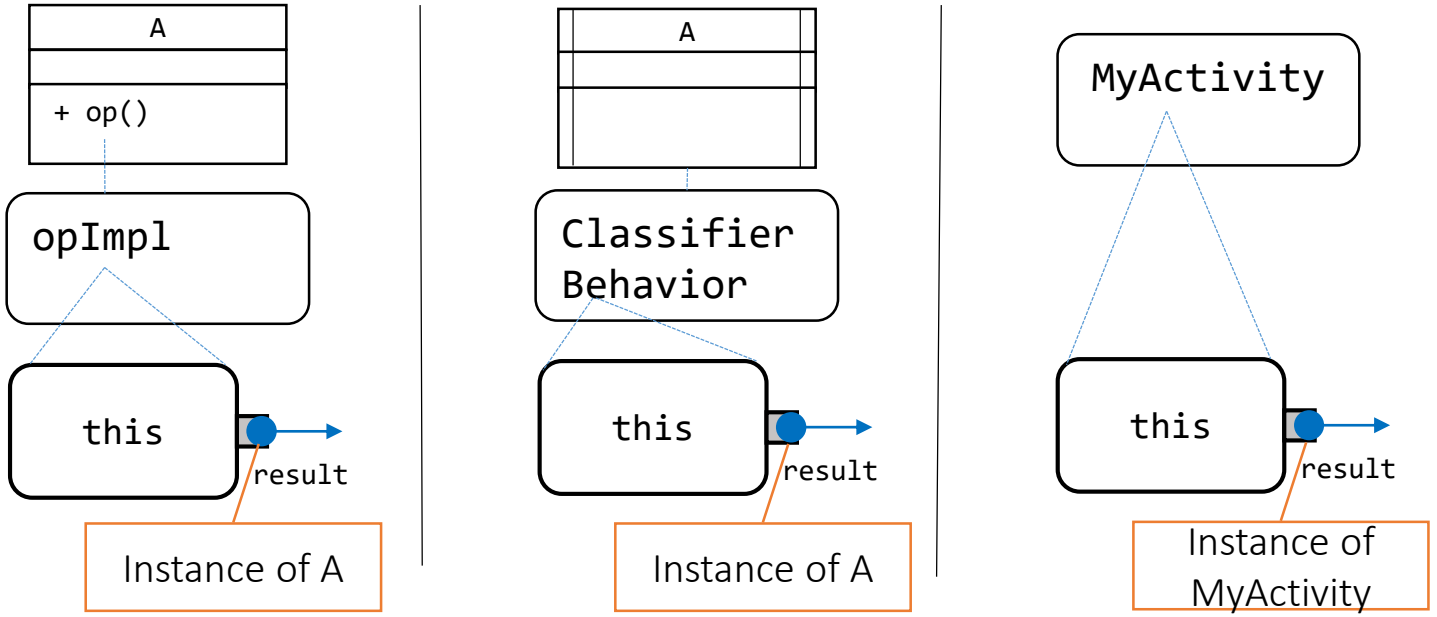
Base action – Test identity



TestIdentityAction

- Assess the equality between the provided values
  - Two values are equal since they have the same types as well the same values for their properties
  - Two enumeration values are equal since they use the same enumeration literal

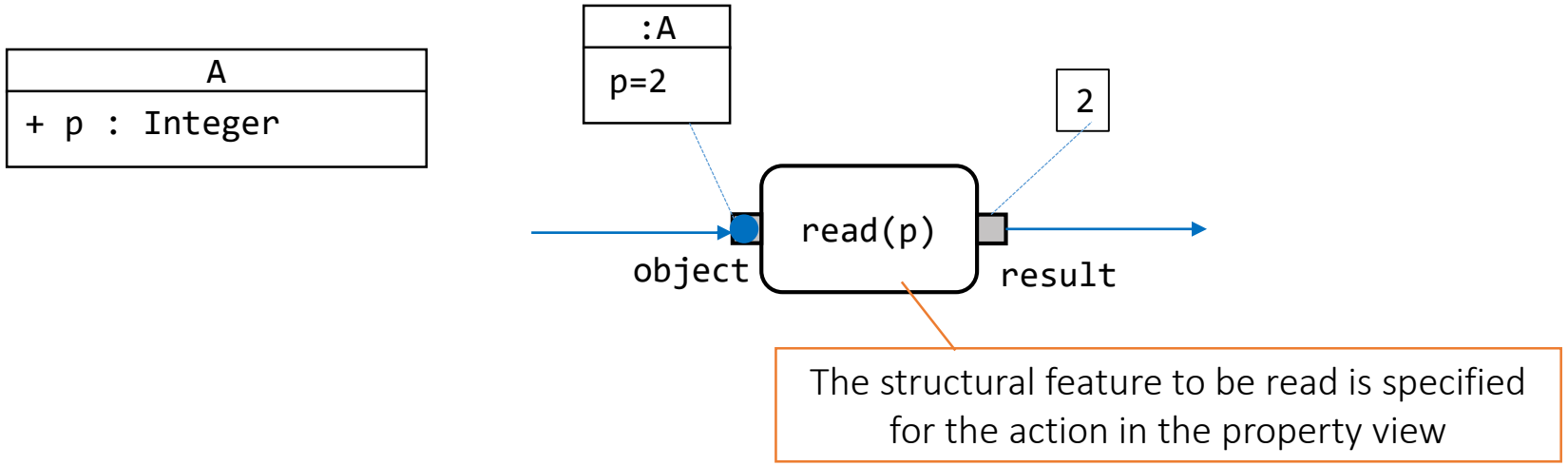
Base action – Read self



Provides a reference on the execution context

- Looks like a “this” instruction in C++ or Java
- If the activity is used in standalone mode
  - The context is the activity itself
- If the activity is used as an implementation for an operation
  - The context is the class instance that was used to make the operation call
- If the activity is used as a classifier behavior
  - The context is the instance of the active class executing the classifier behavior

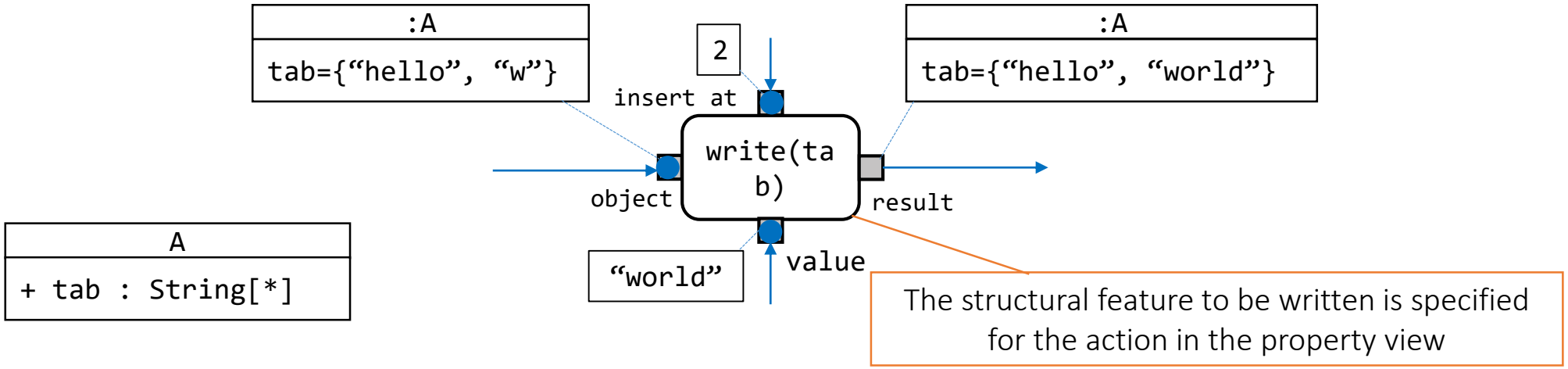
Base action – Read structural feature



Read a structural feature that belongs to a classifier instance

- Requires a target to read
  - Basically the object into which the feature must be read
- Provides the result of the reading on its output pin
  - Type of the value is the type of feature

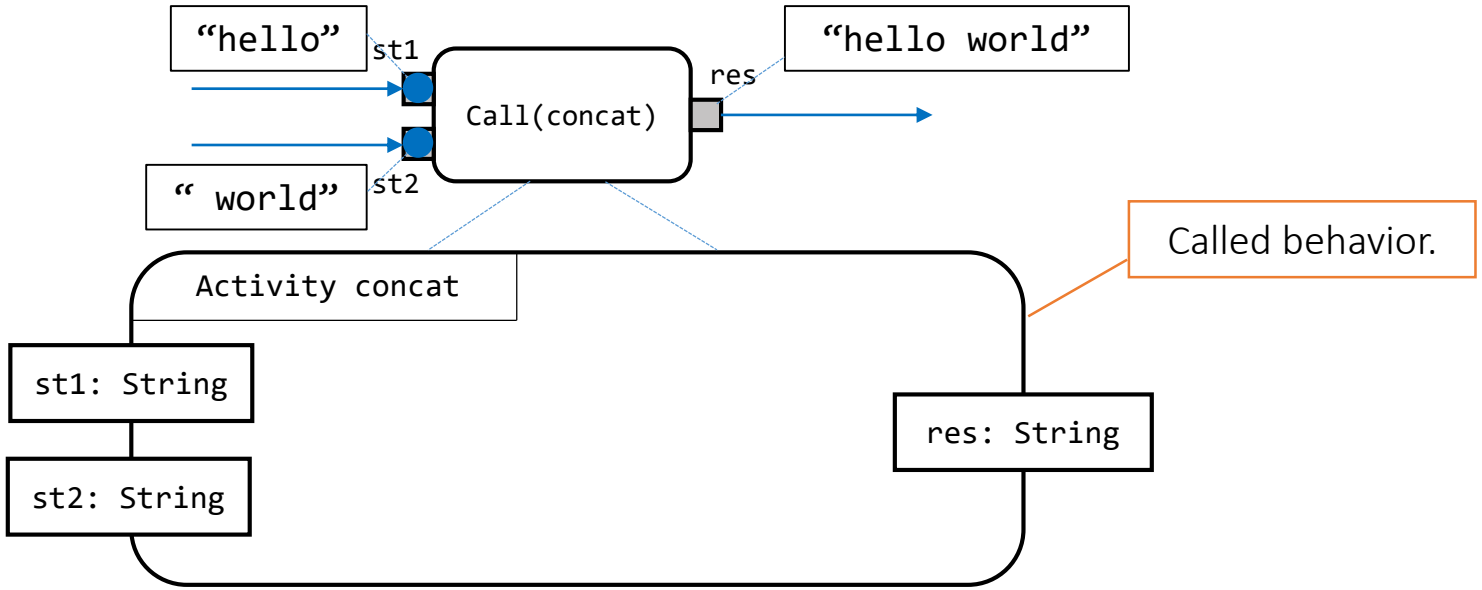
Base action – Add structural feature



Update the value of a structural feature that belongs to a classifier instance

- Requires a target to read
  - Basically the object into which the feature must be modified
- Requires the value to be assigned to the structural feature
  - A value that a type conforming to the structural feature
- Provides the result on its output pin
  - The result is the modified object
- If a multi-valued property is modified then
  - An index can be specified to explicitly state which value must be changed

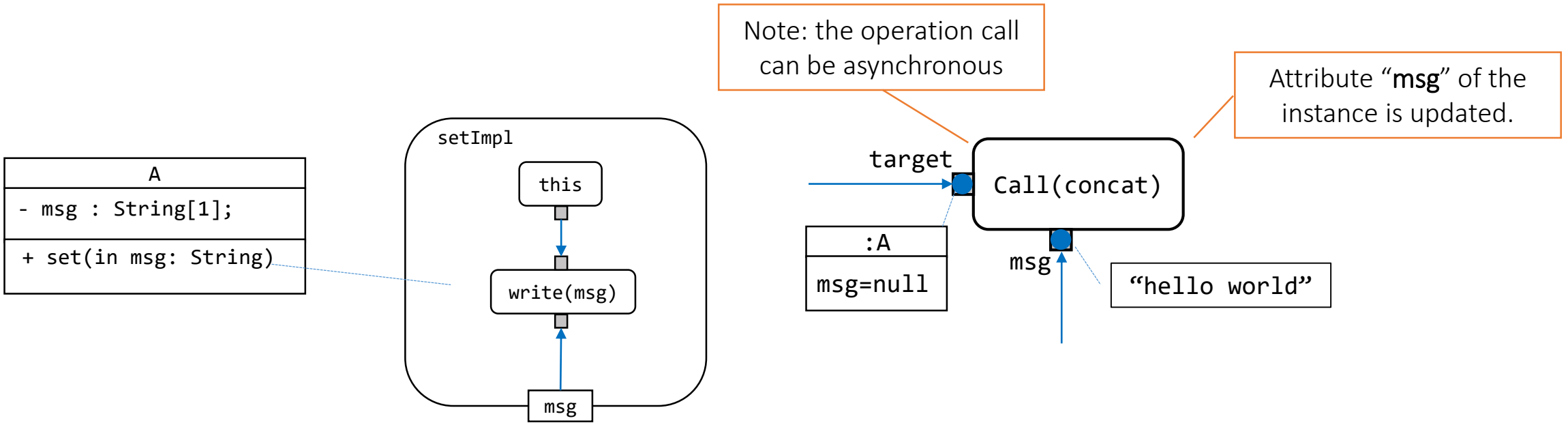
Call behavior action



Call to a specific behavior

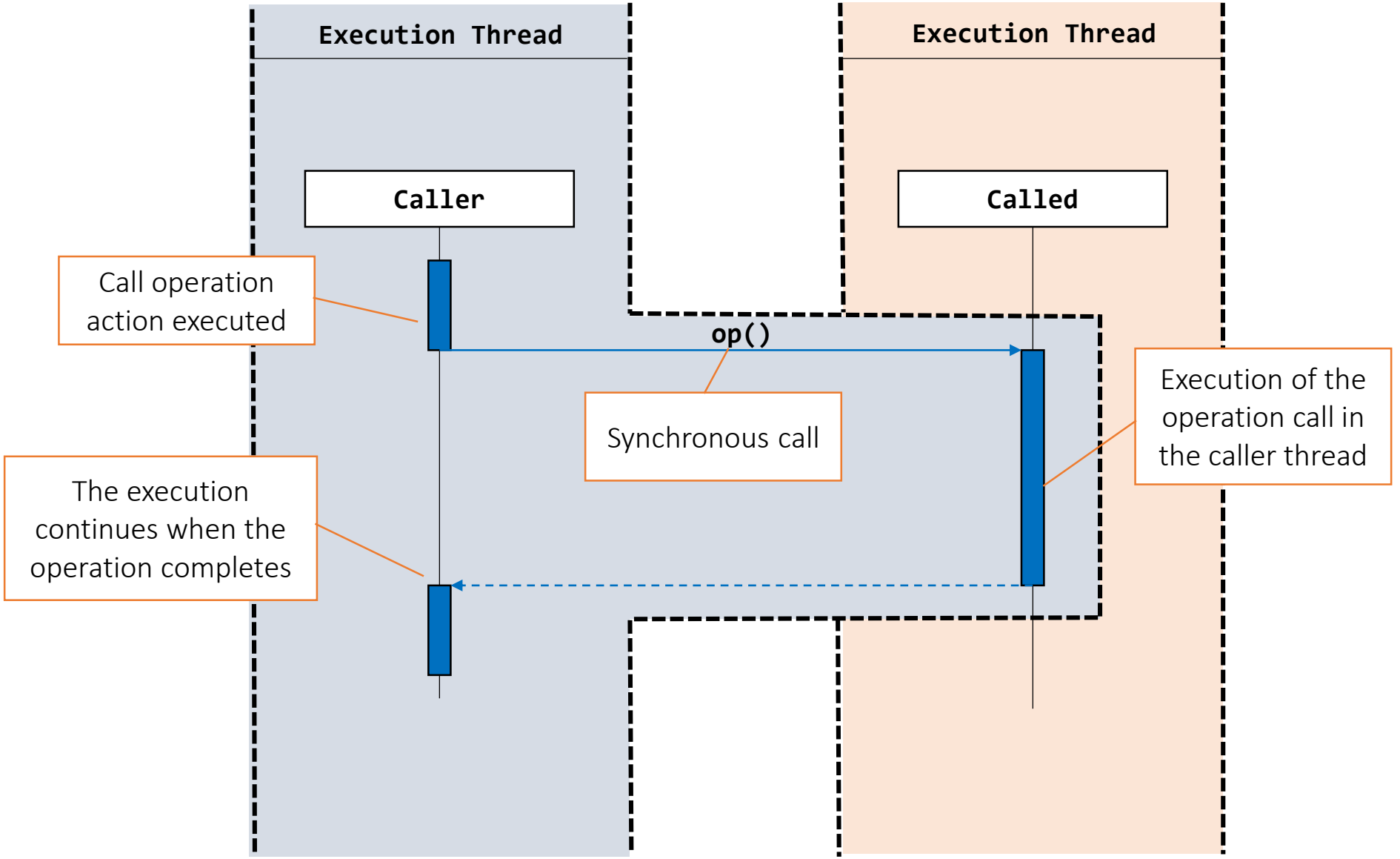
- Similar to a function call in C for example
- Requires the behavior to be called to be specified
  - This can be realized through the property view of Papyrus
- Eventually requires values for input pins if the called behavior as parameters

Call Operation action

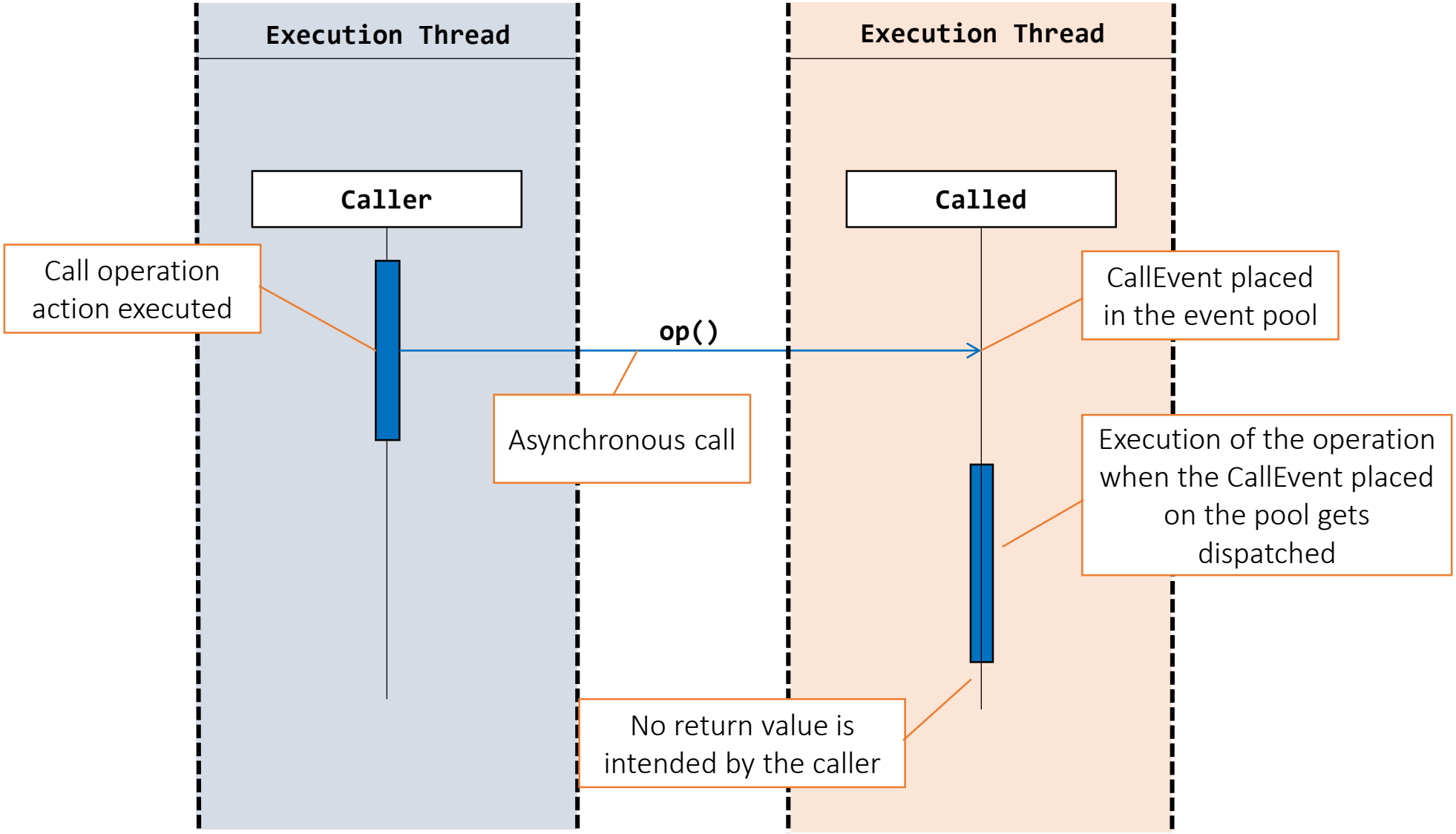


- Requires a target on which the operation can be called – an instance of a classifier
- Requires the operation to be called to be specified
  - This can be realized through the property view of Papyrus
- Eventually requires values for input pins if the called operation has parameters

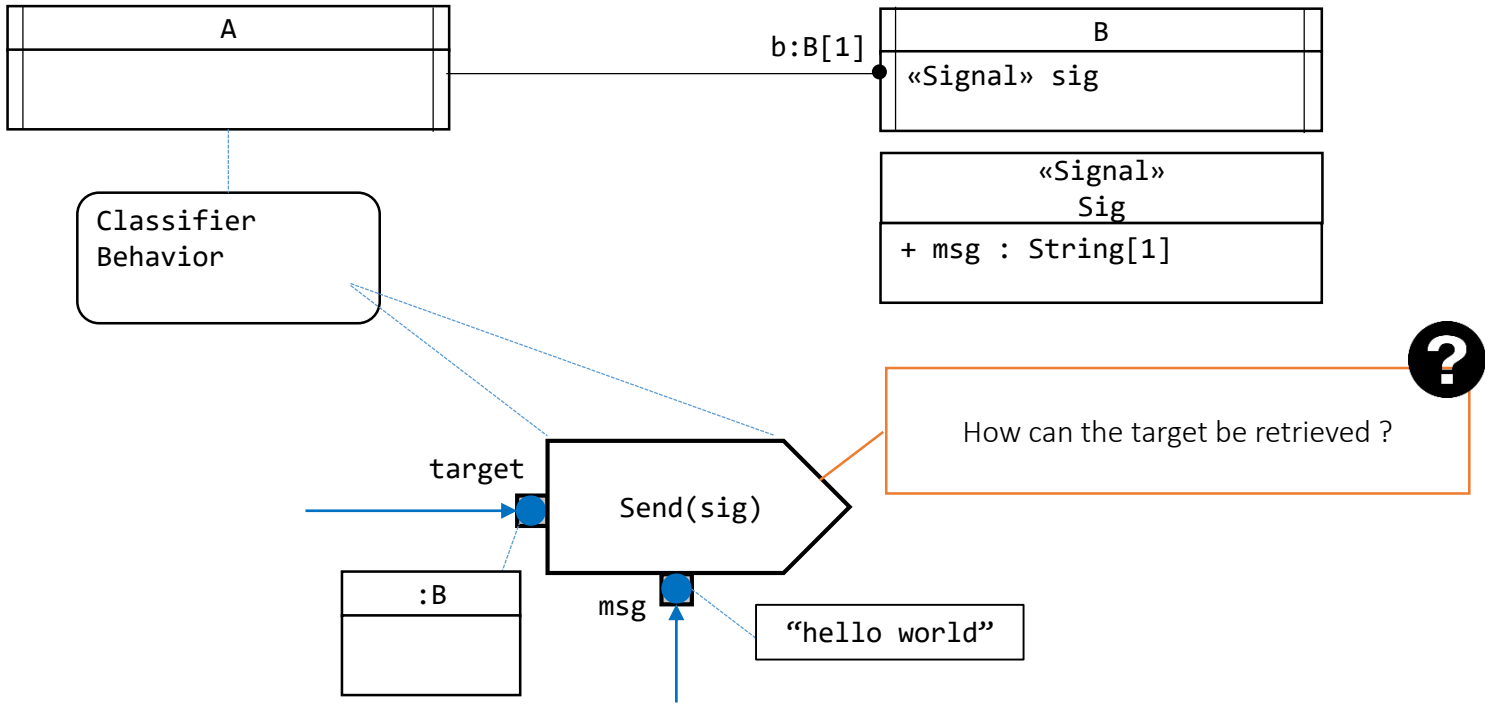
Reminder – Synchronous calls



Reminder – vs Asynchronous calls



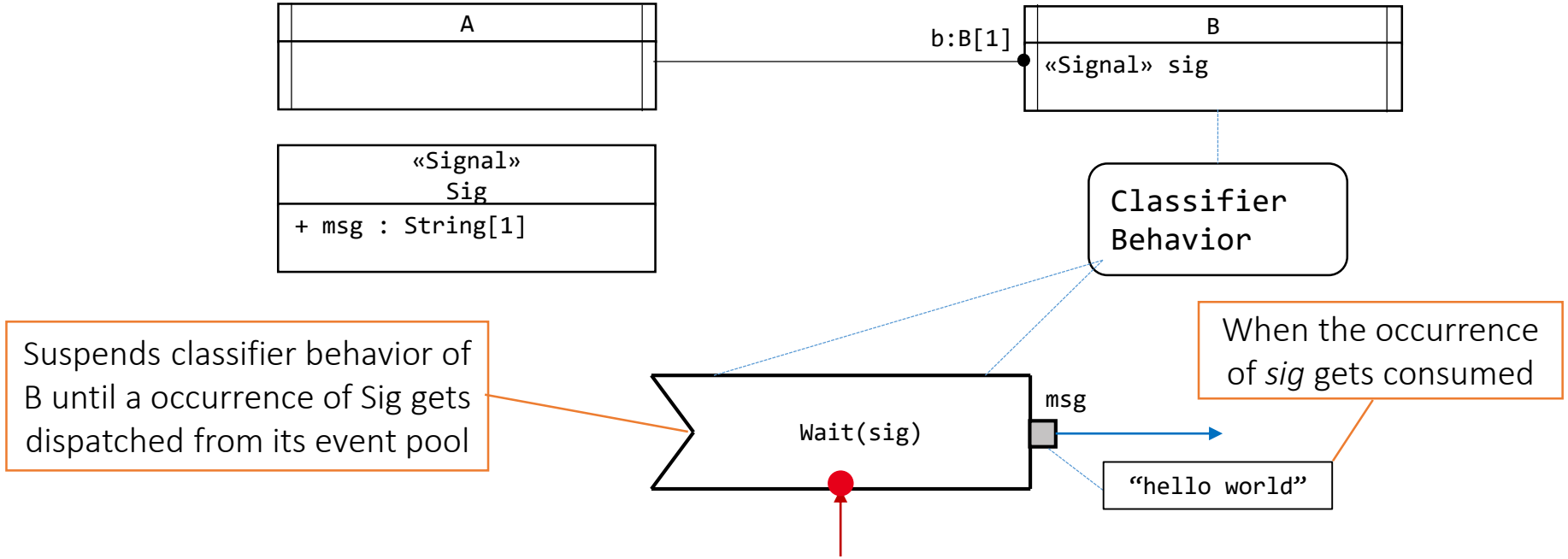
Asynchronous communications (sender side)



Send a signal to an instance of an active object

- Requires a target to which the signal is sent – an instance of an active classifier (with classifier behavior)
- Requires the specification of the signal type to be sent (e.g. via property view of Papyrus)
- Eventually requires values for input pins if the signal that is sent has structural features

Asynchronous communications (receiver side)



Suspends the classifier behavior that executes this action

- Register an event accepter for the event (SignalEvent, CallEvent, …)
  - The one referenced by the trigger attached to the “AcceptEventAction”
- Execution continues when the expected event gets dispatched from the event pool
- If the expected event ships values (e.g., for signal properties)
  - Properties values are placed on output pin(s) of the action
  - Note: this requires the action to specify “isUnmarshall” to true

Event dispatching and run-to-completion (RTC)

Event dispatching

- Semantics is shared for all kind of behaviors
- Events are placed in the pool according to their order of arrival
- An event that is consumed cannot return to the pool
  - Exception for specific situation (e.g., deferred events for state-machines)
- Only one event is dispatched (i.e. consumed) at a time ⇨ start RTC step

Run to completion

- Starts with the consumption of an event
- As long as there is a matched trigger
  - The execution starts from the element to which the trigger is attached.
  - The execution flows propagates as long as possible
  - Step completes, when execution flow stops (e.g., unsatisfied dependencies of an activity)

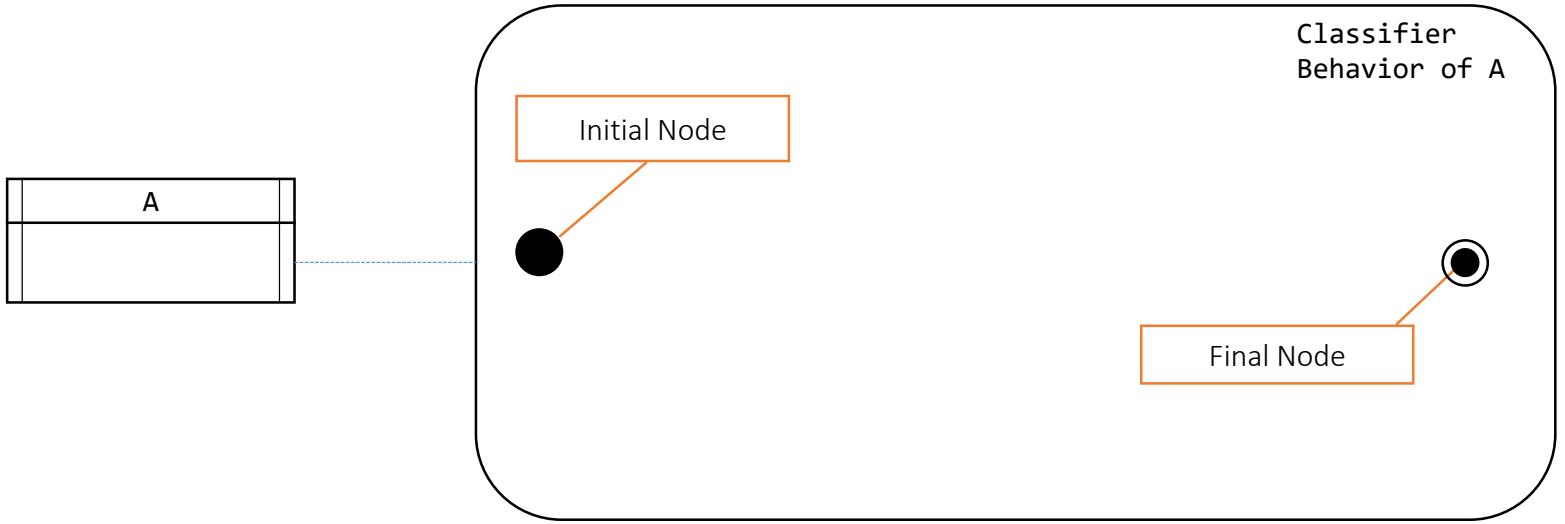
Initial and final nodes

Initial Node – starting point of an activity

- If not specified, all nodes in without incoming edges are started concurrently

Final Node – end point of an activity

- When this node is reached, the containing activity terminates



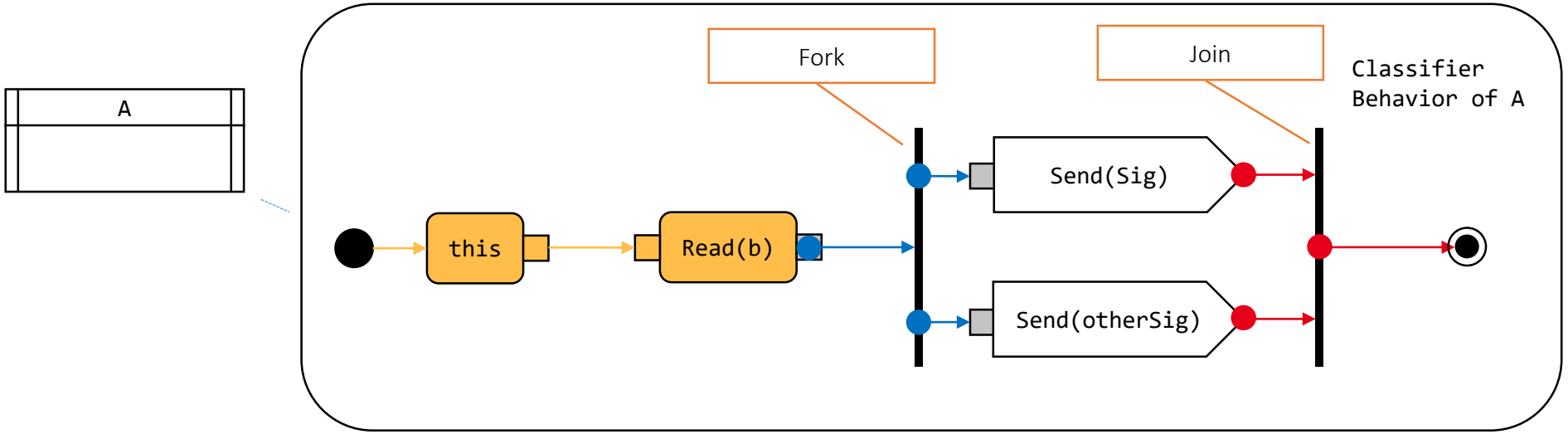
Fork and join – Parallel execution flows

Fork – beginning of multiple concurrent execution flows

- Each outgoing edges of the fork fires concurrently
- Each branch is executed in parallel with the other

Join - Denotes the end multiple concurrent execution flows

- To be traversed the join must receive a token from all of its incoming edges



Merge and Decision – Conditional execution & loops

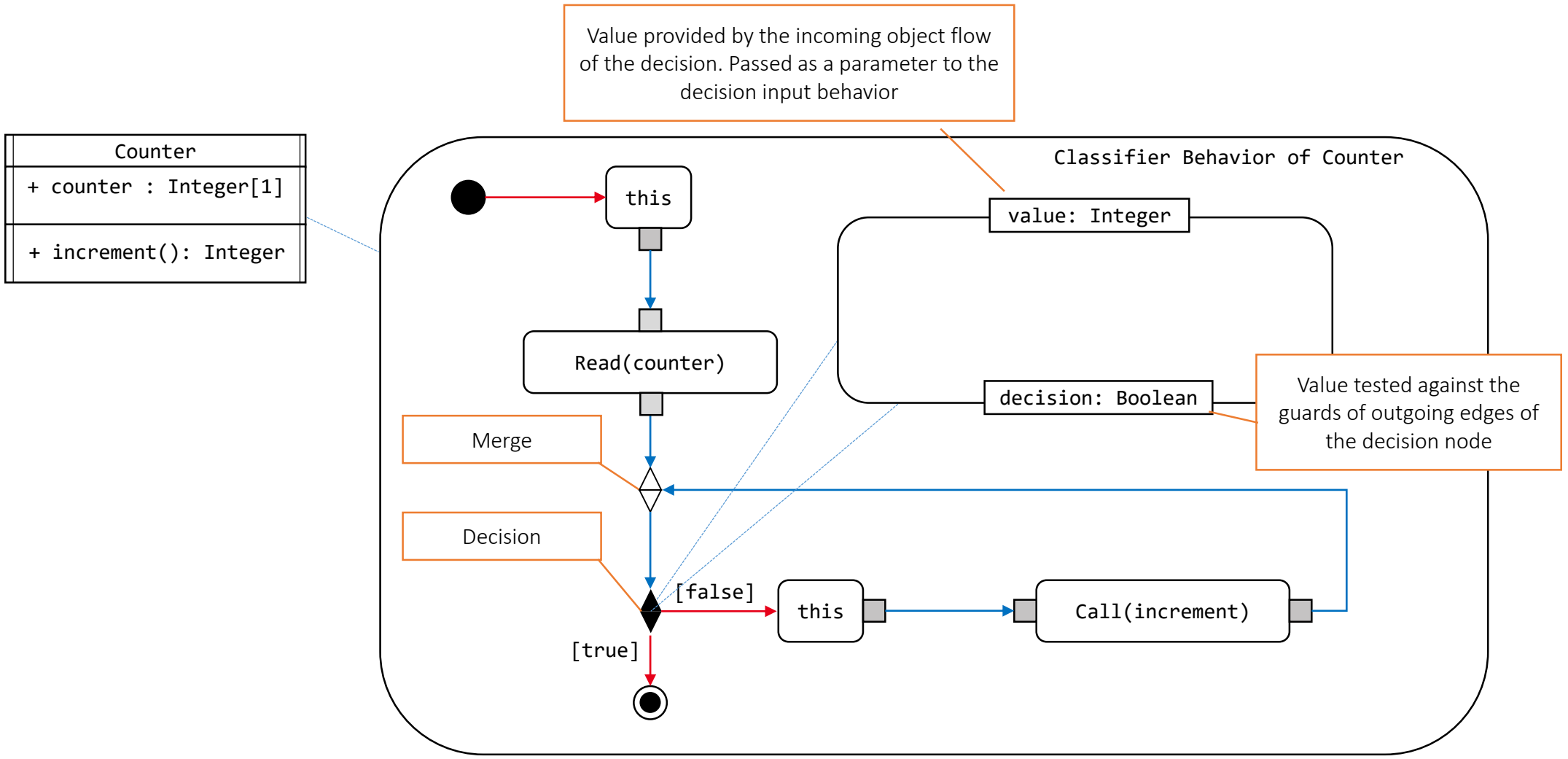
Merge – merge multiple incoming flows into a single one

- If a single incoming edge is traversed then the merge is traversed
- The execution flow continues on the outgoing edge

Decision – route execution flow on a specific path

- Branch choice is based on evaluation of *decision value* against guard specifications on outgoing edges
- Decision value can be
  - Values produced by the decision input behavior
  - Values arriving with incoming object flow if no decision input behavior and no input flow
  - Value arriving by the input flow if there is one but no decision input behavior

Merge and Decision – Conditional execution & loops







Send TPs to ansgar.radermacher@cea.fr



Update sites (release + nightly)
https://download.eclipse.org/modeling/mdt/papyrus/components/moka/
https://ci.eclipse.org/papyrus/job/papyrus-moka-master/lastSuccessfulBuild/artifact/releng/org.eclipse.papyrus.moka.p2/target/repository/